

Tp03 : Étude de cas Yelp

Préparations

Import des librairie

```
In [341... import pandas as pd
from pandas import read_csv, DataFrame
import numpy as np
from utils import create_data_path
import matplotlib.pyplot as plt
from pandas import read_csv, merge, DataFrame
from numpy import nan, floor
from utils import create_data_path, parse_hours
from matplotlib.pyplot import figure, show
from ipywidgets import Dropdown, interact
```

Variable

```
In [342... export_path: str = "./restaurants_features.csv"
days: list[str] = ["lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi", "di
data_set_path: dict[str, str] = {
    "avis" : create_data_path("avis.csv"),
    "categories" : create_data_path("categories.csv"),
    "checkin" : create_data_path("checkin.csv"),
    "conseils" : create_data_path("conseils.csv"),
    "horaires" : create_data_path("horaires.csv"),
    "restaurants" : create_data_path("restaurants.csv"),
    "services" : create_data_path("services.csv"),
    "utilisateurs" : create_data_path("utilisateurs.csv")
}
```

```
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
c:\Users\DCS2003\Desktop\1-cegep\5-session05\5-collecte_interpretation\2-travaux_pratique\2-python\420-514-MV-TP03
```

Charger les tables en Datasets

```
In [343... df_avis      = read_csv(data_set_path['avis'], skipinitialspace=True)
df_categories = read_csv(data_set_path['categories'], skipinitialspace=True)
df_checkin    = read_csv(data_set_path['checkin'], skipinitialspace=True)
df_conseils   = read_csv(data_set_path['conseils'], skipinitialspace=True)
df_horaires   = read_csv(data_set_path['horaires'], skipinitialspace=True)
df_restaurants = read_csv(data_set_path['restaurants'], skipinitialspace=True)
df_services   = read_csv(data_set_path['services'], skipinitialspace=True)
df_utilisateurs = read_csv(data_set_path['utilisateurs'], skipinitialspace=True)
```

Nettoyage des Datasets

Avis

Verifications :

```
In [344... df_avis
```

Out [344...

		avis_id	utilisateur_id	restaurant_id
0	yi0R0Ugj_xUx_Nek0-_Qig	dacAlZ6fTM6mqwW5uxkskg	ikCg8xy5Jlg_NGPx-MSID,	
1	8e9HxxLjjqc9ez5ezzN7iQ	d6xvYpyzcfbF_AZ8vMB7QA	zvO-PJCpNk4fgAVUnExYA,	
2	Z7wgXp98wYB57QdRY3HQ3w	GYNnVehQeXjty0xH7-6Fhw	FxLfqxdYPA6Z85PFKaqLn,	
3	JVcjMhlavKKn3Ult9p9OXA	TpyOT5E16YASd7EWjLQlrw	AakkkTuGZA2KBodKi2_u8,	
4	6BnQwlxRn7ZuWdzninM9sQ	JSrP-dUmLlwZil7Dp3PQ2A	cHdJXLIKNWixBXpDwEGb_,	
...	...	...	...	...
2073680	bzQFrsMDtGIVoeXp5gbNeg	g5K5E9QrxObn6ZC1q8Mb9g	CRVtzesMuwHK-phmS_oja,	
2073681	0sf0k7p1b1krPfcbP62W5A	Ob6ToKjLna3WOZtrl_m0nw	t1P_vLlyJCloEaUVcwHdT,	
2073682	jPCXuSBbl_cPocgoNBhdkg	V6BjjQICDO4q7TT3ZhaPWw	yA6dKNm_zl1ucZCnwW8ZC,	
2073683	zwzO3yPdGbdgGy3-XQ0SWg	PFileCX8wuvi7P-1mCvIjA	a192hdM0_UVCYLwPJv1Qw,	
2073684	OKmOLFwtI-NjofTHAGzv1Q	Ej3mCvjcwVXCSPB_avIfjQ	PkqN5st3zZ_Xce_5QIBtNC,	

2073685 rows × 8 columns



In [345...

```
df_avis.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2073685 entries, 0 to 2073684
Data columns (total 8 columns):
#   Column          Dtype
---  -
0   avis_id         object
1   utilisateur_id  object
2   restaurant_id   object
3   etoiles         float64
4   useful          int64
5   funny           int64
6   cool            int64
7   date            object
dtypes: float64(1), int64(3), object(4)
memory usage: 126.6+ MB
```

In [346... `df_avis.describe(include = "all")`

Out[346...

	avis_id	utilisateur_id	restaurant_id	e
count	2073685	2073685	2073685	2.073685
unique	2073685	748247	34268	
top	yi0R0Ugj_xUx_Nek0-Qig	CxDOIDnH8gp9KXzpBHJYXw	K7IWdNUhCbcnEvl0NhGewg	
freq	1	1618	6887	
mean	NaN	NaN	NaN	3.682247
std	NaN	NaN	NaN	1.376978
min	NaN	NaN	NaN	1.000000
25%	NaN	NaN	NaN	3.000000
50%	NaN	NaN	NaN	4.000000
75%	NaN	NaN	NaN	5.000000
max	NaN	NaN	NaN	5.000000

◀ ▶

In [347... `df_avis.isna().sum()`

Out[347...

```
avis_id          0
utilisateur_id   0
restaurant_id    0
etoiles          0
useful           0
funny            0
cool             0
date             0
dtype: int64
```

```
In [348... df_avis[(df_avis['avis_id'] == '') | (df_avis['utilisateur_id'] == '') | (df_avis['
```

```
Out[348...      avis_id  utilisateur_id  restaurant_id  etoiles  useful  funny  cool  date
```

```
In [349... df_avis[(df_avis['etoiles'] < 1) & (df_avis['etoiles'] > 5)]
```

```
Out[349...      avis_id  utilisateur_id  restaurant_id  etoiles  useful  funny  cool  date
```

```
In [350... df_avis[(df_avis['useful'] < 0) & (df_avis['funny'] < 0) & (df_avis['cool'] < 0)]
```

```
Out[350...      avis_id  utilisateur_id  restaurant_id  etoiles  useful  funny  cool  date
```

Nettoyages :

```
In [351... df_avis['avis_id'] = df_avis['avis_id'].astype('string')
df_avis['restaurant_id'] = df_avis['restaurant_id'].astype('string')
df_avis['utilisateur_id'] = df_avis['utilisateur_id'].astype('string')
df_avis['etoiles'] = df_avis['etoiles'].astype('int64')
df_avis['date'] = pd.to_datetime(df_avis['date'], errors='coerce')
df_avis.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2073685 entries, 0 to 2073684
Data columns (total 8 columns):
#   Column          Dtype
---  -
0   avis_id         string
1   utilisateur_id  string
2   restaurant_id   string
3   etoiles         int64
4   useful         int64
5   funny          int64
6   cool           int64
7   date            datetime64[ns]
dtypes: datetime64[ns](1), int64(4), string(3)
memory usage: 126.6 MB
```

```
In [352... df_avis['date'].isna().sum()
```

```
Out[352... np.int64(0)
```

```
In [353... df_avis = df_avis.drop(columns=['useful', 'funny', 'cool'])
```

Dataset nettoyé :

```
In [354... df_avis
```

Out [354...

	avis_id	utilisateur_id	restaurant_id
0	yi0R0Ugj_xUx_Nek0-_Qig	dacAlZ6fTM6mqwW5uxkskg	ikCg8xy5Jlg_NGPx-MSID,
1	8e9HxxLjjqc9ez5ezzN7iQ	d6xvYpyzcfbF_AZ8vMB7QA	zvO-PJCpNk4fgAVUnExYA,
2	Z7wgXp98wYB57QdRY3HQ3w	GYNnVehQeXjty0xH7-6Fhw	FxLfqxdYPA6Z85PFKaqLn,
3	JVcjMhlavKKn3Ult9p9OXA	TpyOT5E16YASd7EWjLQlrw	AakkkTuGZA2KBodKi2_u8,
4	6BnQwlxRn7ZuWdzninM9sQ	JSrP-dUmLLwZil7Dp3PQ2A	cHdJXLIKNWixBXpDwEGb_
...	...	...	...
2073680	bzQFrsMDtGIVoeXp5gbNeg	g5K5E9QrxObn6ZC1q8Mb9g	CRVtzesMuwHK-phmS_oja,
2073681	0sf0k7p1b1krPfcbP62W5A	Ob6ToKjLna3WOZtrl_m0nw	t1P_vLlyJCloEaUVcwHdT,
2073682	jPCXuSBbl_cPocgoNBhdkg	V6BjjQICDO4q7TT3ZhaPWw	yA6dKNm_zl1ucZCnwW8ZC,
2073683	zwzO3yPdGbdgGy3-XQ0SWg	PFileCX8wuvi7P-1mCvIjA	a192hdM0_UVCYLwPJv1Qw,
2073684	OKmOLFwtI-NjofTHAGzv1Q	Ej3mCvjcwVXCSPB_avIfjQ	PkqN5st3zZ_Xce_5QlBtNC,
2073685 rows × 5 columns			

Categories

Vérifications :

In [355...

df_categories

Out[355...

	restaurant_id	categorie
0	ICwqJWMxvIUQt1Re_tDn4w	Breakfast & Brunch
1	ICwqJWMxvIUQt1Re_tDn4w	Diners
2	ICwqJWMxvIUQt1Re_tDn4w	American (Traditional)
3	pd0v6sOqpLhFJ7mkplaixw	Sandwiches
4	pd0v6sOqpLhFJ7mkplaixw	Gluten-Free
...	...	...
89940	7KlpgRjjAmVabPzxcExs0g	Mexican
89941	0fY-zYyP2fDmp2YXFsuNTg	Sandwiches
89942	0fY-zYyP2fDmp2YXFsuNTg	Soup
89943	0fY-zYyP2fDmp2YXFsuNTg	Bagels
89944	cfrN6-lQC-dzjBtNBjefpQ	American (New)

89945 rows × 2 columns

In [356...

df_categories.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 89945 entries, 0 to 89944
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   restaurant_id    89945 non-null  object
1   categorie         89945 non-null  object
dtypes: object(2)
memory usage: 1.4+ MB
```

In [357...

df_categories.describe(include='all')

Out[357...

	restaurant_id	categorie
count	89945	89945
unique	33816	626
top	QSwpHbPjDhWYKP8qlmlkXg	Nightlife
freq	23	4827

In [358...

df_categories[df_categories.duplicated(subset=['restaurant_id', 'categorie'], keep=

Out[358...

	restaurant_id	categorie
26944	mlkVAm2DkGnKnNVR1AnQLA	Gas Stations
26947	mlkVAm2DkGnKnNVR1AnQLA	Gas Stations

In [359...

```
df_categories.isna().sum()
```

Out[359...

```
restaurant_id    0  
categorie        0  
dtype: int64
```

In [360...

```
df_categories[(df_categories['categorie'] == '') | (df_categories['restaurant_id']
```

Out[360...

restaurant_id	categorie
---------------	-----------

In [361...

```
for categorie in sorted(df_categories['categorie'].unique()):  
    print(categorie)
```


Acai Bowls
Accessories
Accountants
Acne Treatment
Active Life
Acupuncture
Adult Entertainment
Advertising
Afghan
African
Air Duct Cleaning
Airport Lounges
Airport Shuttles
Airports
Amateur Sports Teams
American (New)
American (Traditional)
Amusement Parks
Animal Assisted Therapy
Animal Physical Therapy
Animal Shelters
Antiques
Apartments
Appliances
Appliances & Repair
Aquarium Services
Aquariums
Arabian
Arcades
Argentine
Armenian
Art Classes
Art Galleries
Art Museums
Art Schools
Art Supplies
Arts & Crafts
Arts & Entertainment
Asian Fusion
Australian
Austrian
Auto Customization
Auto Glass Services
Auto Parts & Supplies
Auto Repair
Automotive
Baby Gear & Furniture
Bagels
Bakeries
Bangladeshi
Bankruptcy Law
Bar Crawl
Barbeque
Barbers
Bars
Bartenders

Basque
Beach Bars
Beaches
Beauty & Spas
Bed & Breakfast
Beer
Beer Bar
Beer Garden
Beer Gardens
Beer Hall
Belgian
Beverage Store
Bike Rentals
Bike Repair/Maintenance
Bikes
Bingo Halls
Bistros
Boat Charters
Boat Dealers
Boat Repair
Boating
Bocce Ball
Body Shops
Books
Bookstores
Botanical Gardens
Bowling
Brasseries
Brazilian
Breakfast & Brunch
Breweries
Brewpubs
British
Bubble Tea
Buffets
Building Supplies
Bulgarian
Burgers
Burmese
Bus Tours
Business Consulting
Butcher
CSA
Cabaret
Cafes
Cafeteria
Cajun/Creole
Cambodian
Campgrounds
Canadian (New)
Candy Stores
Cannabis Clinics
Cannabis Collective
Cannabis Dispensaries
Cantonese
Car Dealers

Car Rental
Car Share Services
Car Wash
Car Window Tinting
Cardiologists
Cards & Stationery
Caribbean
Carpet Installation
Casinos
Caterers
Champagne Bars
Check Cashing/Pay-day Loans
Cheese Shops
Cheesesteaks
Chicken Shop
Chicken Wings
Chinese
Chiropractors
Chocolatiers & Shops
Churches
Churros
Cideries
Cinema
Climbing
Clothing Rental
Clowns
Cocktail Bars
Coffee & Tea
Coffee & Tea Supplies
Coffee Roasteries
Coffeeshops
Colleges & Universities
Colombian
Comedy Clubs
Comic Books
Commercial Real Estate
Community Centers
Community Service/Non-Profit
Computers
Contractors
Convenience Stores
Conveyor Belt Sushi
Cooking Classes
Cooking Schools
Cosmetic Dentists
Cosmetic Surgeons
Cosmetics & Beauty Supply
Counseling & Mental Health
Country Clubs
Country Dance Halls
Couriers & Delivery Services
Courthouses
Creperies
Cuban
Cultural Center
Cupcakes

Currency Exchange
Custom Cakes
Czech
Czech/Slovakian
DJs
Damage Restoration
Dance Clubs
Dance Schools
Dance Studios
Day Spas
Delicatessen
Delis
Dentists
Department Stores
Desserts
Diagnostic Services
Dim Sum
Diners
Dinner Theater
Discount Store
Distilleries
Dive Bars
Doctors
Dog Walkers
Dominican
Donairs
Donuts
Door Sales/Installation
Drive-Thru Bars
Drugstores
Dry Cleaning
Dry Cleaning & Laundry
Eastern European
Eatertainment
Education
Egyptian
Electricians
Electronics
Emergency Medicine
Empanadas
Escape Games
Ethical Grocery
Ethiopian
Ethnic Grocery
Event Photography
Event Planning & Services
Eyebrow Services
Eyelash Service
Eyewear & Opticians
Falafel
Farmers Market
Fashion
Feng Shui
Festivals
Filipino
Financial Services

Fire Protection Services
Fireplace Services
Fish & Chips
Fitness & Instruction
Flea Markets
Flooring
Floral Designers
Florists
Flowers & Gifts
Fondue
Foundation Repair
French
Fruits & Veggies
Funeral Services & Cemeteries
Furniture Stores
Game Meat
Gas Stations
Gastropubs
Gay Bars
Gelato
General Dentistry
German
Gift Shops
Glass & Mirrors
Gluten-Free
Go Karts
Golf
Graphic Design
Greek
Grilling Equipment
Grocery
Guamanian
Guest Houses
Gun/Rifle Ranges
Gyms
Hainan
Hair Extensions
Hair Removal
Hair Salons
Hair Stylists
Haitian
Hakka
Halal
Hardware Stores
Hawaiian
Head Shops
Health & Medical
Health Markets
Health Retreats
Heating & Air Conditioning/HVAC
Herbs & Spices
Himalayan/Nepalese
Hobby Shops
Holistic Animal Care
Home & Garden
Home Cleaning

Home Decor
Home Health Care
Home Inspectors
Home Services
Honduran
Hong Kong Style Cafe
Hookah Bars
Horse Racing
Horseback Riding
Hospitals
Hostels
Hot Dogs
Hot Pot
Hot Tub & Pool
Hotel bar
Hotels
Hotels & Travel
Hungarian
IT Services & Computer Repair
Iberian
Ice Cream & Frozen Yogurt
Ice Delivery
Indian
Indonesian
Indoor Playcentre
Interior Design
International
International Grocery
Internet Cafes
Investing
Irish
Irish Pub
Island Pub
Italian
Izakaya
Japanese
Japanese Curry
Jazz & Blues
Jewelry
Juice Bars & Smoothies
Karaoke
Kebab
Keys & Locksmiths
Kids Activities
Kids Hair Salons
Kiosk
Kitchen & Bath
Kombucha
Korean
Kosher
Laboratory Testing
Lakes
Landmarks & Historical Buildings
Laotian
Laser Hair Removal
Laser Tag

Latin American
Laundromat
Laundry Services
Lawyers
Leather Goods
Lebanese
Libraries
Life Coach
Lighting Fixtures & Equipment
Limos
Local Fish Stores
Local Flavor
Local Services
Lounges
Macarons
Magicians
Mags
Makeup Artists
Malaysian
Marinas
Marketing
Martial Arts
Masonry/Concrete
Mass Media
Massage
Massage Therapy
Mattresses
Mauritius
Meat Shops
Medical Centers
Medical Spas
Medical Transportation
Meditation Centers
Mediterranean
Men's Clothing
Metal Fabricators
Mexican
Middle Eastern
Milkshake Bars
Mini Golf
Mobile Phone Repair
Mobile Phones
Modern European
Mongolian
Moroccan
Mosques
Movers
Museums
Music & DVDs
Music & Video
Music Venues
Musical Instruments & Teachers
Musicians
Nail Salons
Nail Technicians
New Mexican Cuisine

Newspapers & Magazines
Nicaraguan
Nightlife
Noodles
Nurseries & Gardening
Nutritionists
Occupational Therapy
Office Cleaning
Office Equipment
Olive Oil
Optometrists
Organic Stores
Orthodontists
Outdoor Furniture Stores
Outlet Stores
Paint & Sip
Paint-Your-Own Pottery
Pakistani
Pan Asian
Parking
Parks
Party & Event Planning
Party Bus Rentals
Party Equipment Rentals
Party Supplies
Pasta Shops
Patisserie/Cake Shop
Pediatricians
Performing Arts
Persian/Iranian
Personal Assistants
Personal Chefs
Personal Shopping
Peruvian
Pet Groomers
Pet Services
Pet Sitting
Pet Stores
Pet Training
Pets
Pharmacy
Photographers
Photography Stores & Services
Physical Therapy
Piano Bars
Piercing
Pita
Pizza
Playgrounds
Plumbing
Poke
Polish
Pool & Billiards
Pool & Hot Tub Service
Pool Halls
Pop-up Shops

Popcorn Shops
Portuguese
Post Offices
Poutineries
Preschools
Pretzels
Print Media
Private Tutors
Professional Services
Property Management
Psychics
Psychologists
Public Markets
Public Services & Government
Public Transportation
Pubs
Puerto Rican
RV Parks
Race Tracks
Ramen
Real Estate
Real Estate Agents
Real Estate Services
Recreation Centers
Reflexology
Rehabilitation Center
Reiki
Religious Organizations
Resorts
Rest Stops
Restaurant Supplies
Rock Climbing
Roofing
Russian
Sailing
Salad
Salvadoran
Sandwiches
Scandinavian
Scottish
Screen Printing
Screen Printing/T-Shirt Printing
Seafood
Seafood Markets
Security Services
Security Systems
Service Stations
Session Photography
Shanghainese
Shared Office Spaces
Shaved Ice
Shaved Snow
Shoe Stores
Shopping
Shopping Centers
Sicilian

Siding
Signmaking
Singaporean
Skating Rinks
Ski Resorts
Ski Schools
Skin Care
Skydiving
Slovakian
Smokehouse
Soba
Soccer
Social Clubs
Software Development
Soup
South African
Southern
Souvenir Shops
Spanish
Speakeasies
Special Education
Specialty Schools
Spiritual Shop
Sporting Goods
Sports Bars
Sports Clubs
Sports Wear
Spray Tanning
Sri Lankan
Stadiums & Arenas
Steakhouses
Street Vendors
Strip Clubs
Studio Taping
Sugar Shacks
Summer Camps
Supernatural Readings
Supper Clubs
Surf Schools
Sushi Bars
Swimming Pools
Swimwear
Synagogues
Syrian
Szechuan
Tabletop Games
Tacos
Tai Chi
Taiwanese
Tanning
Tapas Bars
Tapas/Small Plates
Tasting Classes
Tattoo
Tax Services
Taxis

Tea Rooms
Team Building Activities
Teeth Whitening
Tempura
Tennis
Teppanyaki
Tex-Mex
Thai
Themed Cafes
Thrift Stores
Ticket Sales
Tickets
Tiki Bars
Tires
Tobacco Shops
Tonkatsu
Tours
Towing
Toy Stores
Traditional Clothing
Trainers
Transportation
Travel Services
Trinidadian
Turkish
Tuscan
Udon
Ukrainian
Unofficial Yelp Events
Used
Used Bookstore
Uzbek
Vacation Rentals
Vape Shops
Vegan
Vegetarian
Vehicle Wraps
Venezuelan
Venues & Event Spaces
Videos & Video Game Rental
Vietnamese
Vintage & Consignment
Vinyl Records
Vitamins & Supplements
Waffles
Walking Tours
Water Stores
Waxing
Web Design
Wedding Planning
Weight Loss Centers
Whiskey Bars
Wholesale Stores
Wholesalers
Wigs
Windows Installation

Wine & Spirits
 Wine Bars
 Wine Tasting Classes
 Wine Tasting Room
 Wineries
 Women's Clothing
 Wraps
 Yelp Events
 Yoga
 Zoos

Nettoyage :

```
In [362... df_categories = df_categories.drop_duplicates(subset=['restaurant_id', 'categorie'])
df_categories[df_categories.duplicated(subset=['restaurant_id', 'categorie'], keep=
```

```
Out[362... restaurant_id categorie
```

```
In [363... df_categories['restaurant_id'] = df_categories['restaurant_id'].astype('string')
df_categories['categorie'] = df_categories['categorie'].astype('string')
df_categories.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 89944 entries, 0 to 89944
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   restaurant_id    89944 non-null  string
1   categorie         89944 non-null  string
dtypes: string(2)
memory usage: 2.1 MB
```

Dataset nettoyé :

```
In [364... df_categories
```

Out [364...

	restaurant_id	categorie
0	ICwqJWMxvIUQt1Re_tDn4w	Breakfast & Brunch
1	ICwqJWMxvIUQt1Re_tDn4w	Diners
2	ICwqJWMxvIUQt1Re_tDn4w	American (Traditional)
3	pd0v6sOqpLhFJ7mkplaixw	Sandwiches
4	pd0v6sOqpLhFJ7mkplaixw	Gluten-Free
...	...	...
89940	7KlpgRjjAmVabPzxcExs0g	Mexican
89941	0fY-zYyP2fDmp2YXFsuNTg	Sandwiches
89942	0fY-zYyP2fDmp2YXFsuNTg	Soup
89943	0fY-zYyP2fDmp2YXFsuNTg	Bagels
89944	cfrN6-IQC-dzjBtNBjefpQ	American (New)

89944 rows × 2 columns



Check-in

Vérifications :

In [365...

```
df_checkin
```

Out[365...

	restaurant_id	date
0	--1UhMGODdWsrMastO9DZw	2016-04-26 19:49:16
1	--1UhMGODdWsrMastO9DZw	2016-08-30 18:36:57
2	--1UhMGODdWsrMastO9DZw	2016-10-15 02:45:18
3	--1UhMGODdWsrMastO9DZw	2016-11-18 01:54:50
4	--1UhMGODdWsrMastO9DZw	2017-04-20 18:39:06
...	...	...
5492277	zzzaLBwimxVej4tY6qFOUQ	2014-07-04 21:28:29
5492278	zzzaLBwimxVej4tY6qFOUQ	2014-07-14 01:02:55
5492279	zzzaLBwimxVej4tY6qFOUQ	2014-07-14 21:26:59
5492280	zzzaLBwimxVej4tY6qFOUQ	2014-07-17 01:06:02
5492281	zzzaLBwimxVej4tY6qFOUQ	2014-07-17 19:20:28

5492282 rows × 2 columns

In [366...

df_checkin.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5492282 entries, 0 to 5492281
Data columns (total 2 columns):
#   Column      Dtype
---  -
0   restaurant_id  object
1   date           object
dtypes: object(2)
memory usage: 83.8+ MB
```

In [367...

df_checkin.describe(include='all')

Out[367...

	restaurant_id	date
count	5492282	5492282
unique	32831	5390249
top	DkYS3arLOhA8si5uUEmHOw	2013-09-29 01:59:07
freq	25835	4

In [368...

```
df_checkin[df_checkin['date'] == '2013-12-26 22:01:42']
# Pas le même restaurant, ne sont donc pas des doublons
```

Out[368...

	restaurant_id	date
1061257	BnpuVvPy3sfLlkqKCx6Xnw	2013-12-26 22:01:42
1287581	DwS5_jKur2mfqvqDp_gptg	2013-12-26 22:01:42
1608572	HnbkGxUs7oWKYDeB-Pk82Q	2013-12-26 22:01:42
2146208	NXNa5OYmN1CNmXwHhDxpRA	2013-12-26 22:01:42

In [369...

```
df_checkin.isna().sum()
```

Out[369...

```
restaurant_id    0
date             0
dtype: int64
```

In [370...

```
# Vérification s'il y a des strings vides
df_checkin[(df_checkin['restaurant_id'] == '')]
```

Out[370...

```
restaurant_id  date
```

Nettoyage :

In [371...

```
# Conversion vers les types appropriés
df_checkin['restaurant_id'] = df_checkin['restaurant_id'].astype('string')
df_checkin['date'] = pd.to_datetime(df_checkin['date'], errors='coerce')
df_checkin.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5492282 entries, 0 to 5492281
Data columns (total 2 columns):
#   Column      Dtype
---  -
0   restaurant_id  string
1   date           datetime64[ns]
dtypes: datetime64[ns](1), string(1)
memory usage: 83.8 MB
```

Dataset nettoyé :

In [372...

```
df_checkin
```

Out[372...

	restaurant_id	date
0	--1UhMGODdWsrMastO9DZw	2016-04-26 19:49:16
1	--1UhMGODdWsrMastO9DZw	2016-08-30 18:36:57
2	--1UhMGODdWsrMastO9DZw	2016-10-15 02:45:18
3	--1UhMGODdWsrMastO9DZw	2016-11-18 01:54:50
4	--1UhMGODdWsrMastO9DZw	2017-04-20 18:39:06
...	...	...
5492277	zzzaBwimxVej4tY6qFOUQ	2014-07-04 21:28:29
5492278	zzzaBwimxVej4tY6qFOUQ	2014-07-14 01:02:55
5492279	zzzaBwimxVej4tY6qFOUQ	2014-07-14 21:26:59
5492280	zzzaBwimxVej4tY6qFOUQ	2014-07-17 01:06:02
5492281	zzzaBwimxVej4tY6qFOUQ	2014-07-17 19:20:28

5492282 rows × 2 columns

Conseils

Vérifications :

In [373...

df_conseils

Out [373...

	utilisateur_id	restaurant_id	nb_compliments	date
0	UPw5DWs_b-e2JRBS-t37Ag	VaKXUpmWTTWDKbpJ3aQdMw	0	2017-03-03 03:51
1	jRyO2V1pA4CdVVqCIOPc1Q	5KheTjYPu1HcQzQFtm4_vw	0	2017-12-01 01:46
2	FuTJWFYm4UKqewaoss1KA	TkoyGi8J7YFjA6SbaRzrxg	0	2017-03-21 21:32
3	CaFbX7topxdLpBJ_SNDfGQ	r49iBfbnfoK7yt4rdsL_7g	0	2017-01-06 06:39
4	8zXgNCKusOHMwh5Tj_8yCQ	qrSsS0pk7SL67MP5nN8tlg	0	2017-12-05 05:10
...	...	...	...	...
416798	yx_38-qpzJU2Oqb5P3QBqA	ZeeWNTA3wpabS_0Y1We7DA	0	2017-05-18 18:32
416799	Lu7d-uVVEYZrmxtX0gl8hQ	3GEEy7RP6e4bT4LAIWFMFQ	0	2017-05-14 14:47
416800	cHaCQ03WO3OWNmySffQprw	y-ZZe1dz8AMjiAwNV03h_A	0	2017-06-22 22:14
416801	mkbx55W8B8aPLgDqex7qgg	OMRYQihVjqzjoNoQgyhgQ	2	2017-11-15 15:34
416802	T4R5ZKfcGt1GIAgjCh67eA	n5Hc_iMVkyxcS0SXiBCbOQ	0	2017-10-16 16:57

416803 rows × 4 columns



In [374...

df_conseils.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 416803 entries, 0 to 416802
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   utilisateur_id  416803 non-null  object
1   restaurant_id   416803 non-null  object
2   nb_compliments  416803 non-null  int64
3   date            416803 non-null  object
dtypes: int64(1), object(3)
memory usage: 12.7+ MB
```

In [375... `df_conseils.describe(include='all')`
Pas de compliments négatifs

Out[375...

	utilisateur_id	restaurant_id	nb_compliments	date
count	416803	416803	416803.000000	416803
unique	156743	28827	NaN	416187
top	mkbx55W8B8aPLgDqex7qgg	DkYS3arLOhA8si5uUEmHOw	NaN	2017-07-29 02:22:28
freq	743	1503	NaN	3
mean	NaN	NaN	0.015852	NaN
std	NaN	NaN	0.141101	NaN
min	NaN	NaN	0.000000	NaN
25%	NaN	NaN	0.000000	NaN
50%	NaN	NaN	0.000000	NaN
75%	NaN	NaN	0.000000	NaN
max	NaN	NaN	9.000000	NaN

In [376... `df_conseils[df_conseils['date'] == '2017-07-29 02:22:28']`

Out[376...

	utilisateur_id	restaurant_id	nb_compliments	date
194070	ZKt3JdtmIRdcQI7piOZ18g	MgSd4P3ATkiywbbraFygHg	0	2017-07-29 02:22:28
213089	ZKt3JdtmIRdcQI7piOZ18g	MgSd4P3ATkiywbbraFygHg	0	2017-07-29 02:22:28
216507	ZKt3JdtmIRdcQI7piOZ18g	MgSd4P3ATkiywbbraFygHg	0	2017-07-29 02:22:28

In [377... `df_conseils[df_conseils['date'].duplicated()]`

Out[377...

	utilisateur_id	restaurant_id	nb_compliments	date
6065	zpnDr3EBaVU8QPNHE3eDbg	l6u-Ohml_IUItexEOSCKzg	0	2011-10-06 23:30:29
6805	ia5kvJxYR8gPltI9O4W0Mg	7JR42RkKub9bsBSeXfAjew	0	2015-01-19 01:11:12
7559	rpWCcNQD7nGVUYoU6vzCFw	2UaMR7lOJ3lcXizaDpv7KA	0	2011-10-08 04:15:19
13095	ghwqimFsDVMEUOYVDTdmxg	3GEEy7RP6e4bT4LAIWFMFQ	0	2014-06-20 01:26:10
18329	ZfZCj9pT54yYSRwlzr4CKQ	Q194SAY2nwnSZ18c0oBvDg	0	2012-03-29 22:18:01
...	...	...	...	...
414739	WeqG_Kg7W9REJ8KtjNq_OQ	xKhjhko44Wr7ljMLnsV43g	0	2018-06-15 04:11:01
414933	lddmAHba81zPOUuBmNi3iA	iwzVhM5CirzkNS5wTxwZeA	0	2014-02-22 02:15:53
415682	eet8jFl-FrSbsJfLCFC1EA	I-KFzdnJcqbUHOAdO2I7lg	0	2013-08-09 01:38:17
416100	gGlbzg9b8BO4FKeEygmfhA	SycRVheLdFcK-2jt8j5d4Q	0	2012-06-27 01:36:45
416135	FtdW5kxnDwYZ0h23WUx8Yw	ElH5pDc75v4haOcww-SEJg	0	2017-02-16 18:18:10

616 rows × 4 columns

In [378... `df_conseils[df_conseils.duplicated(subset=['utilisateur_id', 'restaurant_id', 'date`

Out [378...

	utilisateur_id	restaurant_id	nb_compliments	date
1257	rt3PC7WCgCKsoufmQJELfw	CWNMLT- ppaUjLMmrnYDPVg	0	2010-12-14 00:53:04
19300	d_IFklwz3jzt6yRlpTttsQ	DvY_J2gFxudK9HC8kjI3NA	0	2012-06-15 19:10:09
22872	uv5F82-AVISr8-BFswTUWQ	ImfyEVKm3CDGWg18LeRhgA	0	2011-08-06 02:35:08
26309	3CGGvC- FAfo0AMwGVDrkNQ	XXW_OFaYQkkGOGniujZFHg	0	2012-01-10 00:06:16
31396	iffIfwmK6C7Fd7_7luevvQ	84ezTxVxTCml65_UeYRJcA	0	2017-09-24 19:41:03
...	...	...	...	...
411011	4qvl4SkzJDQ-t4hGd0ILLg	_U4M_X9y3Jg2092XjuzSkw	0	2014-11-22 01:20:02
411208	bKB3fzqTMCiHLNvhWAWf1Q	S_F4XmLUZFv9Uzbjy44_NA	0	2017-07-04 21:19:07
412117	bKB3fzqTMCiHLNvhWAWf1Q	S_F4XmLUZFv9Uzbjy44_NA	0	2017-07-04 21:19:07
412538	4qvl4SkzJDQ-t4hGd0ILLg	_U4M_X9y3Jg2092XjuzSkw	0	2014-11-22 01:20:02
413431	A_GGpNX1NBettgTeZWqUrg	X-WXPGYuuknAjZkpYGeRiw	0	2017-10-22 22:36:57

101 rows × 4 columns

In [379...

```
df_conseils[df_conseils.duplicated(subset=['utilisateur_id', 'restaurant_id', 'nb_c
# Beaucoup de doublons trouvés
```

Out[379...

	utilisateur_id	restaurant_id	nb_compliments	date
316741	- LuuXQ2ik3Ra4RiAdWyvdA	i066yR2IDP4FWt6p-k9aFg	0	2014-12-12 04:25:36
319039	- LuuXQ2ik3Ra4RiAdWyvdA	i066yR2IDP4FWt6p-k9aFg	0	2014-12-12 04:25:36
68500	3CGGvC- FAfo0AMwGVDrkNQ	GHS1rVjO-RMcRB6WJLpCDQ	0	2011-11-26 20:49:56
117459	3CGGvC- FAfo0AMwGVDrkNQ	GHS1rVjO-RMcRB6WJLpCDQ	0	2011-11-26 20:49:56
26309	3CGGvC- FAfo0AMwGVDrkNQ	XXW_OFaYQkkGOGniujZFHg	0	2012-01-10 00:06:16
...	...	...	...	...
69696	uv5F82-AVISr8- BFswTUWQ	ImfyEVKm3CDGWg18LeRhgA	0	2011-08-06 02:35:08
214499	wfzJho9yA3lo_DoSlpDiVg	0XEqyl_IVaJNHRKxNcjJZg	0	2014-04-06 00:32:01
229744	wfzJho9yA3lo_DoSlpDiVg	0XEqyl_IVaJNHRKxNcjJZg	0	2014-04-06 00:32:01
64820	y05H_1uQhO4frLTJ5_tGCQ	JTo0oS3iiMI6UPWA6al8SQ	0	2016-02-09 01:29:30
97167	y05H_1uQhO4frLTJ5_tGCQ	JTo0oS3iiMI6UPWA6al8SQ	0	2016-02-09 01:29:30

101 rows × 4 columns

In [380...

df_conseils.isna().sum()

Out[380...

```
utilisateur_id    0
restaurant_id     0
nb_compliments   0
date              0
dtype: int64
```

In [381...

df_conseils[(df_conseils['utilisateur_id'] == '') | (df_conseils['restaurant_id'] =

Out[381... **utilisateur_id restaurant_id nb_compliments date**

Nettoyage :

In [382... *# Suppression des doublons*
`df_conseils = df_conseils.drop_duplicates(subset=['utilisateur_id', 'restaurant_id'])`
`df_conseils[df_conseils.duplicated(subset=['utilisateur_id', 'restaurant_id', 'date'])]`

Out[382... **utilisateur_id restaurant_id nb_compliments date**

In [383... *# Conversion vers les types appropriés*
`df_conseils['utilisateur_id'] = df_conseils['utilisateur_id'].astype('string')`
`df_conseils['restaurant_id'] = df_conseils['restaurant_id'].astype('string')`
`df_conseils['date'] = pd.to_datetime(df_conseils['date'], errors='coerce')`
`df_conseils.info()`

```
<class 'pandas.core.frame.DataFrame'>
Index: 416752 entries, 0 to 416802
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   utilisateur_id  416752 non-null  string
1   restaurant_id   416752 non-null  string
2   nb_compliments  416752 non-null  int64
3   date            416752 non-null  datetime64[ns]
dtypes: datetime64[ns](1), int64(1), string(2)
memory usage: 15.9 MB
```

Dataset nettoyé :

In [384... `df_conseils`

Out [384...

	utilisateur_id	restaurant_id	nb_compliments	date
0	UPw5DWs_b-e2JRBS-t37Ag	VaKXUpmWTTWDKbpJ3aQdMw	0	2017-03-03 03:51
1	jRyO2V1pA4CdVVqCIOPc1Q	5KheTjYPu1HcQzQFtm4_vw	0	2017-12-01 01:46
2	FuTJWFYm4UKqewaoss1KA	TkoyGi8J7YFjA6SbaRzrxg	0	2017-03-21 21:32
3	CaFbX7topxdLpBJ_SNDfGQ	r49iBfbnfoK7yt4rdsL_7g	0	2017-01-06 06:39
4	8zXgNCKusOHMwh5Tj_8yCQ	qrSsS0pk7SL67MP5nN8tlg	0	2017-12-05 05:10
...	...	...	...	...
416798	yx_38-qpzJU2Oqb5P3QBqA	ZeeWNTA3wpabS_0Y1We7DA	0	2017-05-18 18:32
416799	Lu7d-uVVEYZrmxtX0gl8hQ	3GEEy7RP6e4bT4LAIWFMFQ	0	2017-05-14 14:47
416800	cHaCQ03WO3OWNmySffQprw	y-ZZe1dz8AMjiAwNV03h_A	0	2017-06-22 22:14
416801	mkbx55W8B8aPLgDqex7qgg	OMRYQihVjqzjoNoQgyhgQ	2	2017-11-15 15:34
416802	T4R5ZKfcGt1GIAgjCh67eA	n5Hc_iMVkyxcS0SXiBCbOQ	0	2017-10-16 16:57

416752 rows × 4 columns



Horaires

Vérifications :

In [385...

```
df_horaires
# Plusieurs NaN à gérer
```

```
# peut interpréter NaN comme une journée fermée
# sauf si c'est toute la semaine
# données manquantes ou un commerce en faillite?
```

Out[385...

	restaurant_id	lundi	mardi	mercredi	jeudi	vendredi	samedi	dim
0	lCwqJWMxvIUQt1Re_tDn4w	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	
1	pd0v6sOqpLhFJ7mkplaixw	11:0-20:0	11:0-20:0	11:0-20:0	11:0-20:0	11:0-22:0	11:0-22:0	11:0-22:0
2	0vhi__HtC2L4-vScgDFdFw	11:30-22:0	11:30-22:0	11:30-22:0	11:30-22:0	11:30-22:0	12:0-22:0	
3	t65yfB9v9fqIhAkLnnUXdg	11:30-21:0	11:30-21:0	11:30-21:0	11:30-21:0	11:30-21:0	NaN	
4	i7_JPit-2kAbtRTLkic2jA	11:30-22:0	11:30-22:0	11:30-23:0	11:30-23:0	11:30-23:0	NaN	
...	...	...	...	...	...	...	...	...
34263	cjZfgcQwA6KmQ_ANWKN2aw	NaN	5:30-15:0	5:30-15:0	5:30-15:0	5:30-15:0	6:0-15:0	
34264	Hq2edcOTjse7wjK2CwBijQ	11:30-21:0	11:30-21:0	11:30-21:0	11:30-21:0	11:30-22:0	11:30-22:0	
34265	7KlpgRjjAmVabPzxcExs0g	NaN	NaN	NaN	NaN	NaN	NaN	
34266	0fY-zYyP2fDmp2YXFsuNTg	NaN	7:0-14:0	7:0-14:0	7:0-14:0	7:0-14:0	7:0-14:0	
34267	cfrN6-IQC-dzjBtNBjefpQ	NaN	NaN	NaN	NaN	NaN	NaN	

34268 rows × 8 columns



In [386...

df_horaires.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34268 entries, 0 to 34267
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   restaurant_id    34268 non-null  object
1   lundi            22020 non-null  object
2   mardi            23939 non-null  object
3   mercredi         24498 non-null  object
4   jeudi            24654 non-null  object
5   vendredi         24695 non-null  object
6   samedi           23994 non-null  object
7   dimanche         20552 non-null  object
dtypes: object(8)
memory usage: 2.1+ MB
```


In [387... `df_horaires.describe(include='all')`

Out[387...

	restaurant_id	lundi	mardi	mercredi	jeudi	vendredi	samedi	dima
count	34268	22020	23939	24498	24654	24695	23994	2
unique	34268	715	730	744	759	805	826	
top	ICwqJWMxvIUQt1Re_tDn4w	0:0-0:0	11:0-22:0	11:0-22:0	11:0-22:0	11:0-22:0	11:0-22:0	11:0
freq	1	2257	2444	2489	2437	1979	1649	

In [388... `df_horaires[(df_horaires['restaurant_id'] == '')]`

Out[388...

restaurant_id	lundi	mardi	mercredi	jeudi	vendredi	samedi	dimanche
---------------	-------	-------	----------	-------	----------	--------	----------

In [389... `df_horaires.isna().sum()`

Out[389...

restaurant_id	0
lundi	12248
mardi	10329
mercredi	9770
jeudi	9614
vendredi	9573
samedi	10274
dimanche	13716

dtype: int64

In [390...

```
# Vérification du nombre de restaurants sans horaires (fermés?)
jours = ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche']
df_horaires[(df_horaires[jours] == '0:0-0:0').all(axis=1)]
```

Out [390...

	restaurant_id	lundi	mardi	mercredi	jeudi	vendredi	samedi	dimanche
0	ICwqJWMxvIUQt1Re_tDn4w	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
56	hmL0KnYU6F9_egjILmizKA	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
63	QhMhM3Mhokv6V-H0MRuYTg	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
74	qbCPF1Z-Dher2MYUYFCX4Q	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
107	Lyn5sYu1rfxdmntbaOoZJg	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
...	...	...	...	...	...	...	...	...
33701	v6YvYMjxMMS3L-UQe5WAIQ	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
33739	Pnf2HbEcZ4POhBUSUTq6Q	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
33751	68_nloStgS3XUoZiW6doAg	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
33903	tlZmJBQ9UWkaI0Zce_KFhg	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-
33940	BOMgSyi4-TYAnjVn3m0z3A	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-

774 rows × 8 columns



In [391...

```
# Vérification du nombre de restaurants sans horaires (avec des NaN)
df_horaires[(df_horaires[jours].isna()).all(axis=1)]
```

Out[391...

	restaurant_id	lundi	mardi	mercredi	jeudi	vendredi	samedi	dim
5	vMh4madPU3qhNX7P7d8WGA	NaN	NaN	NaN	NaN	NaN	NaN	
8	Xb7jOAa17xtT_uA4sCCAsG	NaN	NaN	NaN	NaN	NaN	NaN	
17	4ScLXRii_WwBn5PbGBI-eg	NaN	NaN	NaN	NaN	NaN	NaN	
18	LAswzVTnT3uCvnKr-SwxEg	NaN	NaN	NaN	NaN	NaN	NaN	
21	wfiKgzubn1cnVUBODsoJ_w	NaN	NaN	NaN	NaN	NaN	NaN	
...	...	...	...	...	...	...	...	
34257	9NUZao5_Jfhj5H45H0ccOQ	NaN	NaN	NaN	NaN	NaN	NaN	
34259	z0nVjbfDw8eb7N57HWNg8w	NaN	NaN	NaN	NaN	NaN	NaN	
34261	ckc3l8cSuggloG4NlquEFQ	NaN	NaN	NaN	NaN	NaN	NaN	
34265	7KlpgRjjAmVabPzxcExs0g	NaN	NaN	NaN	NaN	NaN	NaN	
34267	cfrN6-IQC-dzjBtNBjefpQ	NaN	NaN	NaN	NaN	NaN	NaN	

9420 rows × 8 columns



Nettoyage :

In [392...

```
# Convertir Les NaN en 0:0-0:0 pour signifier une fermeture ou horaire manquant
df_horaires[jours] = df_horaires[jours].fillna('0:0-0:0')
df_horaires[(df_horaires[jours].isna()).all(axis=1)]
```

Out[392...

restaurant_id	lundi	mardi	mercredi	jeudi	vendredi	samedi	dimanche
---------------	-------	-------	----------	-------	----------	--------	----------

In [393...

```
# Conversion vers Le types approprié
df_horaires['restaurant_id'] = df_horaires['restaurant_id'].astype('string')
df_horaires[jours] = df_horaires[jours].astype('string')
df_horaires.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34268 entries, 0 to 34267
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   restaurant_id    34268 non-null  string
1   lundi            34268 non-null  string
2   mardi            34268 non-null  string
3   mercredi         34268 non-null  string
4   jeudi            34268 non-null  string
5   vendredi         34268 non-null  string
6   samedi           34268 non-null  string
7   dimanche         34268 non-null  string
dtypes: string(8)
memory usage: 2.1 MB
```

In [394...

```
df_horaires.isna().sum()
```

Out[394...

```
restaurant_id    0
lundi            0
mardi            0
mercredi         0
jeudi            0
vendredi         0
samedi           0
dimanche         0
dtype: int64
```

Dataset nettoyé :

In [395...

```
df_horaires
```

Out[395...

	restaurant_id	lundi	mardi	mercredi	jeudi	vendredi	samedi	dim
0	ICwqJWMxvIUQt1Re_tDn4w	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	
1	pd0v6sOqpLhFJ7mkplaixw	11:0-20:0	11:0-20:0	11:0-20:0	11:0-20:0	11:0-22:0	11:0-22:0	11:0-22:0
2	0vhi__HtC2L4-vScgDFdFw	11:30-22:0	11:30-22:0	11:30-22:0	11:30-22:0	11:30-22:0	12:0-22:0	
3	t65yfB9v9fqIhAkLnnUXdg	11:30-21:0	11:30-21:0	11:30-21:0	11:30-21:0	11:30-21:0	0:0-0:0	
4	i7_JPit-2kAbtRTLkic2jA	11:30-22:0	11:30-22:0	11:30-23:0	11:30-23:0	11:30-23:0	0:0-0:0	
...	...	...	...	...	...	...	...	...
34263	cjZfgcQwA6KmQ_ANWKN2aw	0:0-0:0	5:30-15:0	5:30-15:0	5:30-15:0	5:30-15:0	6:0-15:0	6:0-15:0
34264	Hq2edcOTjse7wjK2CwBijQ	11:30-21:0	11:30-21:0	11:30-21:0	11:30-21:0	11:30-22:0	11:30-22:0	
34265	7KlpgRjjAmVabPzxcExs0g	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	
34266	0fY-zYyP2fDmp2YXFsuNTg	0:0-0:0	7:0-14:0	7:0-14:0	7:0-14:0	7:0-14:0	7:0-14:0	
34267	cfrN6-IQC-dzjBtNBjefpQ	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	0:0-0:0	

34268 rows × 8 columns



Restaurants

Vérifications :

In [396...

df_restaurants

Out[396...

	restaurant_id	nom	moyenne_etoiles	ville	zone	ferme
0	ICwqJWMxvIUQt1Re_tDn4w	Denny's	2.5	Las Vegas	89110	0
1	pd0v6sOqpLhFJ7mkplaixw	Ike's Love & Sandwiches	4.0	Phoenix	85004	0
2	0vhi_HtC2L4-vScgDFdFw	Midori Japanese Cafe	3.5	Calgary	T2T	0
3	t65yfB9v9fqIhAkLnnUXdg	Pho U	3.5	Toronto	M5A	0
4	i7_JPit-2kAbtRTLkic2jA	John & Sons Oyster House	4.0	Toronto	M5H	0
...	...	...	...	...	...	...
34263	cjZfgcQwA6KmQ_ANWKN2aw	Bruegger's Bagels	3.5	McMurray	15317	1
34264	Hq2edcOTjse7wjK2CwBijQ	Bistro Pointe- Claire	3.5	Pointe- Claire	H9S	1
34265	7KlpgRjjAmVabPzxcExs0g	Taco Mex	4.0	Phoenix	85051	1
34266	0fY-zYyP2fDmp2YXFsuNTg	Gotham Provisions Company	4.0	Sun Prairie	53590	1
34267	cfrN6-IQC-dzjBtNBjefpQ	Rusty Nail Banquet Hall	2.5	Kent	44240	1

34268 rows × 6 columns



In [397...

df_restaurants.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34268 entries, 0 to 34267
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   restaurant_id    34268 non-null  object
1   nom               34268 non-null  object
2   moyenne_etoiles  34268 non-null  float64
3   ville            34268 non-null  object
4   zone             34201 non-null  object
5   ferme            34268 non-null  int64
dtypes: float64(1), int64(1), object(4)
memory usage: 1.6+ MB
```

```
In [398... df_restaurants.describe(include='all')
           # Tous les ID sont uniques
```

```
Out[398...
           restaurant_id      nom  moyenne_etoiles  ville  zone  ferme
count          34268      34268      34268.000000  34268  34201  34268.000000
unique          34268      26024                NaN     614    970                NaN
top  ICwqJWMxvIUQt1Re_tDn4w  McDonald's                NaN  Toronto  89109                NaN
freq              1          382                NaN    4847    573                NaN
mean              NaN          NaN          3.430431     NaN     NaN          0.500000
std              NaN          NaN          0.792022     NaN     NaN          0.500000
min              NaN          NaN          1.000000     NaN     NaN          0.000000
25%              NaN          NaN          3.000000     NaN     NaN          0.000000
50%              NaN          NaN          3.500000     NaN     NaN          0.500000
75%              NaN          NaN          4.000000     NaN     NaN          1.000000
max              NaN          NaN          5.000000     NaN     NaN          1.000000
```

```
In [399... df_restaurants[df_restaurants['restaurant_id'] == '']
```

```
Out[399...
           restaurant_id  nom  moyenne_etoiles  ville  zone  ferme
```

```
In [400... df_restaurants.isna().sum()
```

```
Out[400...
restaurant_id      0
nom                0
moyenne_etoiles    0
ville              0
zone              67
ferme              0
dtype: int64
```

In [401... `df_restaurants[df_restaurants['zone'].isna()]`

Out[401...

	restaurant_id	nom	moyenne_etoiles	ville	zone	ferme
46	NrztAypJHgPcevJNC_VOGg	Bandit Burger	3.0	Calgary	NaN	0
313	dqzdumTXqMvoB7SVXos-Qg	Hodge Podge Truck	3.5	Cleveland	NaN	0
777	USGHcJS_Qpw32ngZEXrG_Q	Tandori Indian Cuisine	1.5	Calgary	NaN	0
1004	ZkT1IR8xBIfnKtApbV96Dg	Nana's Porch	5.0	Charlotte	NaN	0
1081	YBec-htc0Obt3_o2BkRbVA	Caplansky's Thunderin' Thelma	3.0	Toronto	NaN	0
...	...	...	...	...	...	...
23205	qzbbz8QaoPsAlXEsgjtusg	Weekender Cafe	4.0	Whitchurch-Stouffville	NaN	1
24448	m4pZSpO8v84llsGjqHXBaQ	Mobile Waiters	3.5	Phoenix	NaN	1
25534	VRKJpWhQeUE_38jnKrgsrA	ROKs Tacos - Korean BBQ Tacos	3.0	Champaign	NaN	1
29746	iTCH7y2KLRMKMxdEkgUK1g	Street Dogs	4.5	Las Vegas	NaN	1
30448	CCZrbW_8majCaFqni_OasA	Epazote & Haricot	4.5	Saint-Lazare	NaN	1

67 rows × 6 columns

In [402... `# Vérification s'il existe des données autres que 1 ou 0`
`df_restaurants[(df_restaurants['ferme'] != 1) & (df_restaurants['ferme'] != 0)]`

Out[402...

restaurant_id	nom	moyenne_etoiles	ville	zone	ferme
---------------	-----	-----------------	-------	------	-------

In [403... `# Vérification s'il existe des moyennes hors de 1.0 à 5.0`
`df_restaurants[(df_restaurants['moyenne_etoiles'] < 1.0) | (df_restaurants['moyenne_etoiles'] > 5.0)]`

Out[403...

restaurant_id	nom	moyenne_etoiles	ville	zone	ferme
---------------	-----	-----------------	-------	------	-------

In [404... `# Vérifier l'orthographe des noms de ville et si les variantes sont bien de la même`
`for ville in sorted(df_restaurants['ville'].unique()):`

```
zones = df_restaurants[df_restaurants['ville'] == ville]['zone'].unique()
print(ville, list(zones))
```

```
AGINCOURT ['M1W']
Ahwatukee ['85048', '85044']
Airdrie ['T4A', 'T4B']
Ajax ['L1S', 'L1T', 'L1Z']
Akron ['44333', '44313', '44307', '44310', '44321', '44320', '44303']
Alberta ['T2G']
Allentown ['18104']
Allison Park ['15101']
Ambridge ['15003']
Amherst ['44001']
Ange-Gardien ['J0E']
```


Anjou ['H1M', 'H1J']
Anthem ['85086']
Apache Junction ['85220', '85120']
Arnold ['15068']
Aspinwall ['15215']
Aurora ['L4G', '44202', 'L3X', nan]
Avalon ['15202']
Avon ['44011']
Avon Lake ['44012']
Avondale ['85392', '85323', '85329']
Baie-d'Urfe ['H9X']
Baie-d'UrfÃ© ['H9X']
Bainbridge ['44023']
Bainbridge Township ['44023']
Balzac ['T0M']
Bay Village ['44140']
Beachwood ['44122', '44124']
Beaconsfield ['H9W', 'H9H']
Beauharnois ['J6N']
Bedford ['44146', 'J0J']
Bedford Heights ['44128', '44146']
Bedford Hts. ['44146']
Beeton ['L0G']
Belle Vernon ['15012']
Belleville ['53508']
Bellevue ['15202']
Belmont ['28012']
Beloeil ['J3G']
BelÃ©il ['J3G']
Berea ['44017', '44142']
Bethel Park ['15102', '15241', '15234']
Black Earth ['53515']
Blainville ['J7C']
Blakeney ['28277']
Blawnox ['15238']
Bloomfield ['15224']
Bois-des-Filion ['J6Z']
Boisbriand ['J7G', 'J7E', 'J7H']
Bolton ['L7E']
Bond Head ['L0G']
Boucherville ['J4B']
Boulder City ['89005']
Braddock ['15104']
Bradford ['L3Z']
Bradford West Gwillimbury ['L0G', 'L3Z']
Brampton ['L6Y', 'L6T', 'L6W', 'L6X', 'L6S', 'L7A', 'L6P', 'L6R', 'L6V', 'L6Z']
Bratenahl ['44108']
Brecksville ['44141']
Brentwood ['15227']
Bridgeville ['15017']
Broadview Heights ['44147']
Brook Park ['44142', '44135']
Brooklin ['L1M']
Brooklyn ['44144', '44130', '53521']
Brookpark ['44142']
Brossard ['J4Y', 'J4W', 'J4X', 'J4Z', 'J4V']

Brunswick ['44212', '44211']
 Buckeye ['85396', '85326']
 Buena Vista ['15018']
 Burton ['44021']
 Caledon ['L0P', 'L7E', 'L7C', 'L0J']
 Caledon East ['L7C']
 Caledon Village ['L7K']
 Calgary ['T2T', 'T2R', nan, 'T2S', 'T3C', 'T2P', 'T3K', 'T3G', 'T2J', 'T3R', 'T2E',
 'T2B', 'T2M', 'T3J', 'T2H', 'T3A', 'T2K', 'T2N', 'T1Y', 'T2Y', 'T3M', 'T2A', 'T2G',
 'T2L', 'T2V', 'T3H', 'T2Z', 'T2C', 'T3E', 'T2W', 'T2X', 'T3P', 'T3B', 'T3N', 'T3L',
 'T0L', 'T1K']
 Candiac ['J5R']
 Canonsburg ['15317']
 Carefree ['85377']
 Carnegie ['15106', nan]
 Castle Shannon ['15234']
 Catawba Springs ['28037']
 Cave Creek ['85331', '85327']
 Cecil ['15321']
 Central City Village ['85034']
 Centre Island [nan]
 Chagrin Falls ['44022', '44023']
 Chambly ['J3L']
 Champaign ['61820', '61822', '61821', '61824', nan, '61801']
 Champlain ['12919']
 Chandler ['85248', '85226', '85224', '85225', '85249', '85286', '85233', '85295', na
 n]
 Chardon ['44024']
 Chargin Falls ['44022']
 Charlotte ['28208', '28278', '28210', '28262', '28205', '28212', '28204', '28202',
 '28269', '28277', '28227', '28216', '28226', '28273', '28206', '28203', '28213', '28
 215', '28217', '28280', '28282', nan, '28207', '28211', '28105', '28263', '28214',
 '28209', '28270', '28127', '28104', '28246', '28027', '28075', '28078', '28134', '28
 288', '28266', '28285']
 Chateaugay ['J6J', 'J6K']
 Chertsey ['J0K']
 Chesterland ['44026']
 Chestermere ['T1X']
 Cheswick ['15024']
 Chomedey, Laval ['H7V']
 Church Cove ['TR12']
 ChÃtœauquay ['J6K']
 Clairton ['15025']
 Clarkson ['L5J', 'L5K']
 Cleveland ['44113', '44109', '44114', '44135', '44106', '44130', nan, '44126', '4412
 5', '44124', '44134', '44111', '44118', '44115', '44102', '44112', '44144', '44108',
 '44122', '44104', '44127', '44128', '44110', '44129', '44107', '44120', '44103', '44
 121', '44105', '44143', '44142', '44119', '44178', '44117', '44136', '44123', '4419
 5', '44116', '44146']
 Cleveland Heights ['44118', '44115', '44106', '44121']
 Cleveland Hghts. ['44106']
 Clover ['29710']
 Columbia Station ['44028']
 Concord ['28027', 'L4K', '28025', '44077', '44060', '28173']
 Concord Mills ['28027']
 Concord Twp ['44060']

Copley ['44321']
 Coraopolis ['15108']
 Cornelius ['28031']
 Cote-Saint-Luc ['H4W']
 Cottage Grove ['53527']
 Crafton ['15205']
 Cramerton ['28032']
 Creighton ['15030']
 Cross Plains ['53528']
 Cuddy ['15031']
 Cuyahoga Falls ['44223', '44221', '44319']
 Cuyahoga Fls ['44221', '44223']
 C  te Saint-Luc ['H3X']
 C  te-Saint-Luc ['H4W']
 Dallas ['28034', '75247']
 Davidson ['28036', '28031', '27262']
 De Forest ['53532']
 DeForest ['53532']
 Delson ['J5B']
 Denver ['28037']
 Division No. 6 ['T1Y']
 Dollard-Des Ormeaux ['H9G']
 Dollard-Des-Ormeaux ['H8Y', 'H9G', 'H9B']
 Dollard-des Ormeaux ['H9G']
 Dollard-des-Ormeaux ['H9B', 'H9G', 'H8Y', 'H9H']
 Dorval ['H9S', 'H9P', 'H4Y', 'H9R', 'H9B']
 Dravosburg ['15034']
 ETOBICOKE ['M9A']
 East Ajax ['L1S']
 East Cleveland ['44112']
 East Gwillimbury ['L0G', 'L9N', 'L3Y']
 East Liberty ['15206']
 East Pittsburgh ['15112']
 East York ['M4J', 'M4C', 'M4A', 'M4K', 'M4G', 'M4B']
 Eastlake ['44095']
 El Mirage ['85335']
 Elizabeth ['15037']
 Elrama ['15038']
 Elyria ['44035']
 Emsworth ['15202']
 Enterprise ['89113', '89178']
 Est  rel ['J0T']
 Etibicoke ['M9C']
 Etna ['15223']
 Etobicoke ['M8Z', 'M8X', 'M8V', 'M9C', 'M9B', 'M9A', 'M9W', 'M8W', 'M9P', 'M9V', 'M8Y', 'M9R']
 Etobiicoke ['M8V']
 Euclid ['44123', '44117', '44119', '44132']
 Fairlawn ['44333', '44313']
 Fairport Harbor ['44077']
 Fairview Park ['44126', '44116', '44107']
 Finleyville ['15332']
 Fisher ['61843']
 Fitchburg ['53719', '53711', '53713']
 Forest Hills ['15221']
 Fort Mill ['29708']

Fort McDowell ['85264']
Fort Mcdowell ['85264']
Fort Mill ['29707', '29708', '29715', nan, '29716']
Fountain Hills ['85268', '85310']
Ft. Mill ['29708']
Garfield Heights ['44125']
Gastonia ['28052', '28056', '28054']
Gates Mills ['44040']
Gelndale ['85305']
Georgetown ['L7G']
Gibsonia ['15044']
Gifford ['61866', '61847']
Gilbert ['85296', '85295', '85233', '85234', '85297', '85236', '84295', '85298', '85284', '85208', '85206', '85324']
Glassport ['15045']
Glendale ['85302', '85305', '85301', '85306', '85310', '85308', '85303', '85307', '85304', '85318', '85032', '85311', '85020']
Glendale Az ['85310']
Glenshaw ['15116']
Godmanchester ['J0S']
Goodwood ['L0C']
Goodyear ['85338', '85395', '85394']
Grafton ['44044']
Grand River ['44045']
Green Valley ['89123']
Greenfield Park ['J4V']
Guadalupe ['85283']
Halton Hills ['L7G', 'L0P']
Hampton Township ['15101']
Harrisburg ['28075']
Harrison City ['15636']
Heidelberg ['15106']
Hemmingford ['J0L']
Henderson ['89074', '89015', '89052', '89014', '89002', '89011', '89012', '89044', '89123', '89104', nan, '89502', '89183', '89077', '89009']
Herminie ['15637']
Highland Heights ['44143']
Higley ['85236']
Hinckley ['44233']
Homer ['61849']
Homestead ['15120']
Hudson ['44236', 'J7T', 'J0P', '34667']
Huntersville ['28078']
Huntingdon ['J0S']
Hyland Heights ['44143']
Iberville ['J2X']
Imperial ['15126']
Independence ['44131']
Indian Land ['29707']
Indian Trail ['28079', '28110', '28104']
Indian land ['29707']
Inglewood ['L7C']
Irwin ['15642']
Jefferson Hills ['15025']
Joliette ['J6E']
Kahnawake ['J0L']

Kannapolis ['28081', '28083', '28027']
 Kent ['44240']
 King ['L7B', 'L0G']
 King City ['L7B']
 Kirkland ['H9H', 'H9J', 'H9W']
 Kirtland ['44094']
 Kleinburg ['L0J']
 L'ile-Perrot ['J7V']
 L'Äžle-Perrot ['J7V']
 La Prairie ['J5R']
 La Salle ['H8N']
 LaGrange ['44050']
 LaSalle ['H8N']
 Lachine ['H8S', 'H8T', 'H8R']
 Lachute ['J8H']
 Lacolle ['J0J']
 Lagrange ['44050']
 Lake Park ['28079']
 Lake Wylie ['29710']
 Lakewood ['44107']
 Lancaster ['29720']
 Las Vegas ['89110', '89149', '89145', '89103', '89119', '89107', '89117', '89148',
 '89147', '89109', '89128', '89130', '89104', '89106', '89146', '89118', '89102', '89
 183', '89178', '89101', '89169', '89123', '89113', '89191', '89135', '89139', '8913
 6', '89122', '89193', '89121', '89129', '89158', '89156', '89108', '89141', '89166',
 '89142', '89115', '89120', '89019', '89032', '89134', '89143', '89044', '89152', '89
 131', '89030', '89124', '87701', '89155', '89154', '89144', nan, '89084', '89111',
 '89014', '89031', '88901', '89133', '89054', '89173', '89105', '89162', '89138', '89
 018', '89150', '89039', '89199', '89052']
 Lasalle ['H8N', 'H8P', 'H8R']
 Laval ['H7T', 'H7V', 'H7E', 'H7L', 'H7M', 'H7X', 'H7N', 'H7C', 'H7G', 'H7S', 'H7W',
 'H7R', 'H7P']
 Laval, Pont Viau ['H7G']
 Lavaltrie ['J5T']
 Laveen ['85339']
 Laveen Village ['85339']
 Lawrenceville ['15201']
 Leeds ['LS27']
 Leetsdale ['15056']
 Library ['15129']
 Lindale ['44111']
 Litchfield ['44253']
 Litchfield Park ['85340']
 Locust ['28097', '28001']
 Longueuil ['J4H', 'J4N', 'J4J', 'J3Y', 'J4K', 'J4L', 'J4V']
 Lorain ['44053', '44052', '44055']
 Lowell ['28056', '28098']
 Lower Burrell ['15068']
 Lower Lawrenceville ['15201']
 Lyndhurst ['44124', '44122']
 MESA ['85204']
 Macedonia ['44056', '44067']
 Madison ['53704', '53703', '53714', '53719', '53711', '53702', '53715', '53716', na
 n, '53713', '53705', '53718', '53726', '53717', '53562', '53706', '53774', '53175',
 '53593']
 Mahomet ['61853']

Mantua ['44255']
Maple ['L6A', 'L4K', 'L4J']
Maple Heights ['44137']
Maricopa ['85255']
Markham ['L3R', 'L6E', 'L3P', 'L3T', 'L3X', 'L6C', 'L6G', 'L6B', 'L3S', 'L4B', 'L3B', 'L0H', 'M1V', 'L4J']
Marshall ['53559']
Mascouche ['J7K']
Matthews ['28105']
Matthews ['28104', '28105', '28277', '28106']
Mayfield ['44040', '44143']
Mayfield Heights ['44124']
Mayfield Village ['44143']
Mc Donald ['15057']
Mc Farland ['53558']
Mc Kees Rocks ['15136']
Mc Murray ['15317']
McAdenville ['28101']
McCandless Township ['15237']
McDonald ['15057']
McFarland ['53558']
McKees Rocks ['15136']
McKeesport ['15133', '15131', '15132', '15135']
McMasterville ['J3G']
McMurray ['15317', '15301']
McFarland ['53558']
McKees Rocks ['15136']
McKeesport ['15135']
McKnight ['15237']
McMurray ['15317']
Medina ['44256']
Medina Township ['44256']
Mentor ['44060']
Mentor On The Lake ['44060', '44092']
Mentor On the Lake ['44060']
Mentor-on-the-Lake ['44060']
Mercier ['J6R']
Mesa ['85215', '85203', '85202', '85201', '85204', '85206', '85213', '85210', '85205', '85209', '85207', '85208', '85212', nan, '85254', '85233', '85013']
Middleburg Heights ['44130']
Middlefield ['44062']
Middleton ['53562', '53717']
Midland ['28107']
Midnapore ['T2X']
Millvale ['15209']
Milton ['L0P', 'L9T', nan]
Mint Hill ['28227']
Mint Hill ['28227']
Mirabel ['J7J']
Missisauga ['L5L']
Mississauga ['L5M', 'L4X', 'L5N', 'L4W', 'L5R', 'L4Y', 'L5B', 'L5A', 'L5T', 'L5C', 'L4V', 'L4Z', 'L5K', 'L5J', 'L5V', 'L6T', 'L5L', 'L5G', 'L5P', 'M1J', 'L5W', 'L4T', 'L5S', 'L5E', 'L5H', 'L5T1Y7', 'L1M']
Mississauga ['L5A']
Monona ['53716', '53713']
Monongahela ['15063']

Monroe ['28110', '28025']
 Monroeville ['15146']
 Mont St-hilaire ['J3G']
 Mont-Royal ['H3P', 'H3R', 'H4P']
 Mont-Saint-GrÃ©goire ['J0J']
 Mont-Saint-Hilaire ['J3H']
 Monticello ['61856']
 Montreal ['H2T', 'H4A', 'H2X', 'H3G', 'H2Z', 'H2Y', 'H3A', 'H1S', nan, 'H2J', 'H3H',
 'H3N', 'H3B', 'H2L', 'H4C', 'H2W', 'H4B', 'H3J', 'H2V']
 Montreal-Nord ['H1H', 'H1G']
 Montreal-Ouest ['H4X']
 Montreal-West ['H4X']
 Montrose ['44333']
 MontrÃ©al ['H3G', 'H1S', 'H5A', 'H2H', 'H2Z', 'H2X', 'H2L', 'H2W', 'H1E', 'H4A', 'H9
 H', 'H2Y', 'H2V', 'H2K', 'H4B', 'H4P', 'H3B', 'H3C', 'H3S', 'H3K', 'H3L', 'H3N', 'H2
 J', 'H3T', 'H8S', 'H5B', 'H1T', 'H3H', 'H1N', 'H1V', 'H3A', 'H2S', 'H3X', 'H4C', 'H2
 R', 'H2G', 'H2T', 'H4G', nan, 'H1M', 'H3W', 'H4N', 'H1X', 'H2E', 'H4V', 'H1Y', 'H4
 S', 'H1W', 'H3J', 'H4R', 'H1R', 'H3Z', 'H4H', 'H3V', 'H4L', 'H4K', 'H4E', 'H4J', 'H1
 Z', 'H2C', 'H3E', 'H2P', 'H4T', 'H1P', 'H8Y', 'H8N', 'H2B', 'H1L', 'H1H', 'H2M', 'H3
 M', 'H2A', 'H2N', 'H1G', 'H4M', 'H9S', 'H8Z']
 MontrÃ©al-Ouest ['H4X']
 MontÃ©al ['H2L']
 Moon ['15108']
 Moon Township ['15108']
 Mooresville ['28117']
 Moreland Hills ['44022']
 Morgan ['15064']
 Moseley ['23120']
 Mount Albert ['L0G']
 Mount Holly ['28120', '28012']
 Mount Horeb ['53572']
 Mount Lebanon ['15228', '15241']
 Mount Oliver ['15210']
 Munhall ['15120']
 Munroe Falls ['44262']
 Murrys ville ['15668', '15632']
 N Las Vegas ['89030', '89031']
 N Ridgeville ['44039']
 N. Las Vegas ['89030']
 NELLIS AFB ['89115']
 Nellis AFB ['89191']
 Nellis Air Force Base ['89191']
 Neville Island ['15225']
 New Eagle ['15067']
 New Kensington ['15068']
 New Tecumseth ['L0G']
 Newbury ['44065']
 Newmarket ['L3Y', 'L3X']
 Nobleton ['L0G']
 North York ['M2N']
 North Braddock ['15112']
 North Huntingdon ['15642']
 North Las Vegas ['89032', '89084', '89030', '89031', '89081', '89130', '89115', '890
 86']
 North Olmstead ['44070']
 North Olmsted ['44070']

North Randall ['44128']
 North Ridgeville ['44039']
 North Royalton ['44133']
 North Strabane Township ['15317']
 North Toronto ['M5M']
 North Versailles ['15137']
 North York ['M3H', 'M3J', 'M2N', 'M5M', 'M2R', 'M6A', 'M4A', 'M2M', 'M2K', 'M6L', 'M2H', 'L5V', 'M9N', 'M3C', 'M9L', 'M9M', 'M3K', 'M3N', 'M2J', 'M3B', 'M4N', 'M3M', 'M6B', 'M3A', 'M2L', 'M4Y']
 Northfield ['44067']
 Northfield Center ['44067']
 Norton ['44203']
 Oak Ridges ['L4E']
 Oakdale ['15071']
 Oakmont ['15139']
 Oakville ['L6H', 'L6J', 'L6M']
 Oka ['J0N']
 Oldham ['OL2']
 Olmsted Falls ['44138']
 Olmsted Township ['44138']
 Orange Village ['44122']
 Oregon ['53575']
 Ormstown ['J0S']
 Oshawa ['L1H']
 Outremont ['H2V']
 PHOENIX ['85053', '85021', '85083', '85028']
 Pahrump ['89048']
 Painesville ['44077']
 Palgrave ['L7E']
 Paoli ['53508']
 Paradise ['89183', '89109', '89169', '89119']
 Paradise Valley ['85253', '85032']
 Parma ['44129', '44130', '44134', '44131']
 Parma Heights ['44130']
 Paw Creek ['28130']
 Peninsula ['44264']
 Penn Hills ['15235', '15147']
 Penn Hills Township [nan]
 Peoria ['85383', '85381', '85382', '85345', '85305', '85310', '85385']
 Pepper Pike ['44122']
 Perry ['44081']
 Peters Township ['15317']
 Pheonix ['85050', '85004']
 Pheonix AZ ['85041']
 Philo ['61864']
 Phoenix ['85004', '85085', '85007', '85020', '85027', '85022', '85016', '85033', '85048', '85041', '85014', '85035', '85058', '85018', '85051', '85054', '85017', '85032', '85019', '85021', '85029', '85053', '85050', '85028', '85023', '85043', '85034', '85044', '85037', '85031', '85303', '85251', '85040', '85003', '85015', '85013', '85042', '85006', '85086', '85012', '85024', '85009', '85008', '85339', '85201', '85064', nan, '85281', '85001', '85353', '85308', '85310', '85226', '85045', '85073', '85083', '85234', '85304', '85224', '85209', '85305']
 Phx ['85003']
 Pickering ['L1V', 'L1W', 'L1X']
 Piedmont ['J0R']
 Pierrefonds ['H9H', 'H8Y', 'H8Z', 'H9A', 'H9K']

Pincourt ['J7V']
Pineville ['28134', '28226']
Pitcairn ['15140']
Pittsburgh ['15202', '15212', '15209', '15275', '15201', '15237', '15213', '15222', '15203', '15218', '15238', '15221', '15226', '15234', '15232', '15219', '15207', '15217', '15216', '15223', '15235', '15227', '15231', '15236', '15224', '15220', '15122', '15206', '15205', '15210', '15241', '15239', '15233', '15228', '15229', '15211', nan, '15208', '15215', '15044', '15243', '15289', '15136', '15277', '15126', '15120']
Pleasant Hills ['15236']
Plum ['15239']
Pointe-Calumet ['J0N']
Pointe-Claire ['H9R', 'H9B', 'H9S', 'H9G']
Pointe-aux-Trembles ['H1A']
Port Credit ['L5G']
PrÃ©vost ['J0R']
Queen Creek ['85142', '85242']
Ranlo ['28054']
Rantoul ['61866']
Ravenna ['44266']
Rawdon ['J0K']
Regent Square ['15218']
Regional Municipality of York ['L4C']
Repentigny ['J6A']
Rexdale ['M9W']
Richfield ['44286']
Richmond Height ['44143']
Richmond Heights ['44143']
Richmond Hil ['L4C']
Richmond Hill ['L4C', 'L4B', 'L4E', 'L4S', 'L3T', 'L3R', 'L4J']
Richmond Hts ['44143']
Robinson Township ['15205', '15136']
Rock Hill ['29732', '29730']
Rocky River ['44116']
Rocky View ['T4A']
Rocky View County ['T4A']
Rocky View No. 44 ['T0M']
Rosemere ['J7A']
RosemÃ©re ['J7A']
Ross ['15237']
Ross Township ['15237']
Rouses Point ['12979']
Roxboro ['H8Y']
Russellton ['15076']
Sagamore Hills ['44067']
Saint Joseph ['61873']
Saint-Basile-Le-Grand ['J3N']
Saint-Bruno ['J3V']
Saint-Bruno-de-Montarville ['J3V']
Saint-Charles-Borromee ['J6E']
Saint-Constant ['J5A']
Saint-Eustache ['J7R', 'J7P']
Saint-Henri ['H4C']
Saint-Hubert ['J3Y', 'J4T']
Saint-Hyacinthe ['J2S']
Saint-Jean-Sur-Richelieu ['J3B']

Saint-Jean-sur-Richelieu ['J3B', 'J2W', 'J3A']
 Saint-Jerome ['J7Z']
 Saint-JÃ©rÃ©me ['J7Z', 'J7Y']
 Saint-Lambert ['J4P', 'J4S', 'J4R']
 Saint-Laurent ['H4L', 'H4R', 'H4T', 'H4N', 'H4K', 'H4M']
 Saint-Lazare ['J7T', nan]
 Saint-Leonard ['H1S', 'H1R']
 Saint-LÃ©onard ['H1P', 'H1R']
 Saint-Marc-sur-Richelieu ['J0L']
 Saint-Pierre-de-VÃ©ronne-Ã -Pike-River ['J0J']
 Saint-Sauveur ['J0R']
 Saint-Sauveur-des-Monts ['J0R']
 Sainte-Adele ['J8B']
 Sainte-AdÃ¨le ['J8B']
 Sainte-Anne-De-Bellevue ['H9X']
 Sainte-Anne-de-Bellevue ['H9X']
 Sainte-Catherine ['J5C']
 Sainte-DorothÃ©e ['H7Z']
 Sainte-Genevieve ['H9H']
 Sainte-GeneviÃ¨ve ['H9H']
 Sainte-Julie ['J3E']
 Sainte-Marthe ['J0P']
 Sainte-Rose ['H7L']
 Sainte-Therese ['J7E']
 Sainte-ThÃ©rÃ¨se ['J7E']
 Salaberry-De-Valleyfield ['J6T']
 Sauk City ['53583']
 Savoy ['61874']
 Scarborough ['M1L', 'M1S', 'M1K', 'M1R', 'M1V', 'M1T', 'M1H', 'M1N', 'M1J', 'M1B',
 'M1P', 'M4C', 'M1E', 'M1M', 'M1W', 'M1G', 'M1C', 'M4B', 'M6A', 'M1X', 'L6E']
 Scarobrough ['M1S']
 Schomberg ['L0L', 'L0G']
 Schottsdale ['85251']
 Scottsdale ['85251', '85256', '85253', '85255', '85258', '85260', '85257', '85259',
 '85254', '85266', '85250', '85262', '85377', '85264']
 Sedona ['86336']
 Seven Hills ['44131']
 Sewickley ['15143', '15233']
 Shadyside ['15232']
 Shaker Heights ['44120', '44122', '44118']
 Shaler Township ['15101']
 Sharon ['L0G']
 Sheffield ['44035']
 Sheffield Lake ['44054']
 Sheffield Village ['44035', '44054']
 Shorewood Hills ['53705']
 Solon ['44139', '44219', '44022']
 South Amherst ['44001']
 South Euclid ['44118', '44121']
 South Las Vegas ['89118']
 South Park ['15129']
 Spring Hill City View ['15212']
 Spring Valley ['89121', '89147', '89102', '89146', '89103']
 Springdale ['15144']
 Squirrel Hill ['15217']
 St Leonard ['H1R']

St-Jerome ['J7Y']
 St-Laurent ['H4L']
 St-Leonard ['H1P', 'H1S']
 Stallings ['28104']
 Stanley ['28164']
 Ste-Therese-de-Blainville ['J7E']
 Stouffville ['L4A']
 Stoughton ['53589']
 Stow ['44224', '44221']
 Streetsboro ['44241']
 Streetsville ['L5M']
 Strongsville ['44136', '44149', '44133']
 Sturgeon ['15082']
 Summerlin ['89135']
 Summerlin South ['89147']
 Sun City ['85351', '85345']
 Sun City West ['85375']
 Sun Lakes ['85248']
 Sun Praiie ['53590']
 Sun Prairie ['53590']
 Sunrise Manor ['89110']
 Surprise ['85374', '85388', '85379', '85378', '85375', '85387']
 Sutersville ['15083']
 Swissvale ['15218']
 THORNHILL ['L4J']
 TORONTO ['M3B']
 Tallmadge ['44278']
 Tarentum ['15084']
 Tega Cay ['29708']
 Tempe ['85281', '85283', '85282', '85284', '85280', '85287']
 Terrebonne ['J6W', 'J6V', 'J6X']
 Thornhil ['L3T']
 Thornhill ['L3T', 'L4J', 'L6A']
 Tolleson ['85353']
 Toronto ['M5A', 'M5H', 'M9W', 'M1P', 'M4R', 'M9V', 'M4M', 'M6P', 'M4K', 'M2M', 'M2N', 'M4Y', 'M5E', 'M5V', 'M5K', 'M9B', 'M5J', 'M6J', 'M5S', 'M4T', 'M6K', 'M4V', 'M1L', 'M1V', 'M5R', 'M5T', 'M5B', 'L5T', 'M6G', 'M5C', 'M6H', 'M1R', 'M4N', 'M2J', 'M5G', 'M3H', 'M6R', 'M7A', 'M8V', 'M4J', 'M1W', 'M4E', 'M3J', 'M4S', 'M1S', 'M6E', 'M4L', 'M4C', 'M1T', 'M5P', 'M1H', 'M2K', 'M3B', 'M6C', nan, 'M6N', 'M3K', 'M4W', 'M4P', 'M1M', 'M5M', 'M9C', 'M6A', 'M3C', 'M1B', 'M6S', 'M8X', 'M6B', 'M5X', 'M1J', 'M8Z', 'M8W', 'M4B', 'M1E', 'M1C', 'M6M', 'M4G', 'M8Y', 'M1N', 'M1K', 'M9P', 'M9N', 'M9A', 'M2P', 'M1X', 'M5N', 'M4X', 'M5L', 'M4A', 'M6L', 'M3L', 'M9M', 'M3M', 'L3R', 'M3N', 'M1G', 'M5W', 'M4H', 'M3A', 'M2H', 'M9L', 'M2R', 'L6A', 'W8M']
 Tottenham ['L0G']
 Township of Concord ['44060']
 Trafford ['15085']
 Tremont ['44113']
 Troy Township ['44021']
 Turtle Creek ['15145']
 Twinsburg ['44087']
 Unionville ['L3R']
 University Heights ['44118', '44121']
 Upper Saint Clair ['15241']
 Upper St Clair ['15241']
 Urbana ['61801', '61802', '61822', '61803']
 Uxbridge ['L9P']

```

Val-Morin ['J0T']
Valley City ['44280']
Valley View ['44125']
Vaudreuil ['J7V']
Vaudreuil-Dorion ['J7V']
Vaughan ['L6A', 'L4H', 'L4K', 'L0J', 'L4J', 'L4L', 'L1K', 'L7B', 'L3T']
Venetia ['15367']
Venise-en-QuÃ©bec ['J0J']
Verdun ['H4G', 'H3E', 'H4H']
Verona ['53593', '53719', '15147']
Villa Grove ['61956']
Ville Mont-Royal ['H3R']
Wadsworth ['44281']
Walton Hills ['44146']
Warrensville Heights ['44122', '44128']
Waunakee ['53597']
Waxhaw ['28173']
Weddington ['28104']
Wesley Chapel ['28104']
West Elizabeth ['15088']
West Homestead ['15120']
West Mifflin ['15122', '15123']
West View ['15229']
Westlake ['44145']
Westmount ['H3Z', 'H4A']
Westport ['28037']
Wexford ['15090']
Whitby ['L1N', 'L1R', 'L1M', 'L1P']
Whitchurch-Stouffville ['L0H', 'L4A', nan]
White Oak ['15131']
Wickliffe ['44092', '44095']
Wilkens Township ['15235']
Wilkins Township ['15235']
Wilkinsburg ['15221']
Willoughby ['44094', '44096']
Willoughby Hills ['44094', '44092']
Willowdale ['M2N']
Willowick ['44095']
Wilmerding ['15148']
Windsor ['53598']
Woodbridge ['L4L', 'L4H', 'L4K']
Woodmere ['44122']
York ['M9N', 'M6E', 'M6M', 'M6B', 'M6C', 'M6S', 'M6N', '29745']
Youngtown ['85363']
kirtland ['44094']
south Euclid ['44121']
springdale ['15144']
ÃŽle-des-Soeurs ['H3E']

```

Nettoyage :

In [405...

```

# Suppression de colonne non-utilisée
df_restaurants = df_restaurants.drop(columns=['zone'])

```

In [406...

```
# Conversion vers le types approprié
df_restaurants['restaurant_id'] = df_restaurants['restaurant_id'].astype('string')
df_restaurants['nom'] = df_restaurants['nom'].astype('string')
df_restaurants['ville'] = df_restaurants['ville'].astype('string')
df_restaurants['ferme'] = df_restaurants['ferme'].astype(bool)
df_restaurants.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34268 entries, 0 to 34267
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   restaurant_id         34268 non-null  string
1   nom                   34268 non-null  string
2   moyenne_etoiles       34268 non-null  float64
3   ville                 34268 non-null  string
4   ferme                 34268 non-null  bool
dtypes: bool(1), float64(1), string(3)
memory usage: 1.1 MB
```

In [407...

```
# Première Lettre des noms de ville en majuscule
df_restaurants['ville'] = df_restaurants['ville'].str.title()
```

In [408...

```
# Corriger le nom de villes
corrections_villes = {
    "Baie-D'Urfä@": "Baie-D'Urfe",
    'Bainbridge Township': 'Bainbridge',
    'Bedford Heights': 'Bedford',
    'Bedford Hts.': 'Bedford',
    'Belâ"il': 'Beloeil',
    'Bradford West Gwillimbury': 'Bradford',
    'Brookpark': 'Brook Park',
    'Caledon East': 'Caledon',
    'Caledon Village': 'Caledon',
    'Chomedey, Laval': 'Laval',
    'Châ¢Teauguay': 'Chateauguay',
    'Cleveland Heights': 'Cleveland',
    'Cleveland Hghts.': 'Cleveland',
    'Concord Mills': 'Concord',
    'Concord Twp': 'Concord',
    'Cuyahoga Falls': 'Cuyahoga',
    'Că'Te Saint-Luc': 'Cote-Saint-Luc',
    'Că'Te-Saint-Luc': 'Cote-Saint-Luc',
    'De Forest': 'DeForest',
    'Deforest': 'DeForest',
    'Dollard-Des Ormeaux': 'Dollard-Des-Ormeaux',
    'Estã@Rel': 'Estérel',
    'Etibicoke': 'Etobicoke',
    'Etobiicoke': 'Etobicoke',
    'Fort Mill': 'Fort Mill',
    'Ft. Mill': 'Fort Mill',
    'Gelndale': 'Glendale',
    'Glendale Az': 'Glendale',
    'King': 'King City',
    "L'Äžle-Perrot": "L'Ile-Perrot",
```

```

'La Salle': 'LaSalle',
'Lasalle': 'LaSalle',
'Laval, Pont Viau': 'Laval',
'Laveen Village': 'Laveen',
'Mattews': 'Matthews',
'Mayfield': 'Mayfield Village',
'Mc Donald': 'McDonald',
'Mc Farland': 'McFarland',
'Mc Kees Rocks': 'McKees Rocks',
'Mc Murray': 'McMurray',
'Mcadenville': 'McAdenville',
'Mccandless Township': 'McCandless',
'Mcdonald': 'McDonald',
'Mcfarland': 'McFarland',
'Mckees Rocks': 'McKees Rocks',
'Mckeesport': 'McKeesport',
'Mcknight': 'McKnight',
'Mcmasterville': 'McMasterville',
'Mcmurray': 'McMurray',
'Medina Township': 'Medina',
'Mentor On The Lake': 'Mentor',
'Mentor-On-The-Lake': 'Mentor',
'Mint Hill': 'Mint Hill',
'Mississauga': 'Mississauga',
'Mont St-Hilaire': 'Mont-Saint-Hilaire',
'Mont-Saint-Grã@Goire': 'Mont-Saint-Grégoire',
'Montreal-West': 'Montreal-Ouest',
'Montrã@Al': 'Montreal',
'Montrã@Al-Ouest' : 'Montreal-Ouest',
'Montã@Al': 'Montreal',
'Moon Township': 'Moon',
'N Las Vegas': 'Las Vegas',
'N. Las Vegas': 'Las Vegas',
'N Ridgeville': 'North Ridgeville',
'Nellis Afb': 'Nellis Air Force Base',
'North York': 'North York',
'North Las Vegas': 'Las Vegas',
'North Olmstead': 'North Olmsted',
'Northfield Center': 'Northfield',
'Olmsted Township': 'Olmsted Falls',
'Parma': 'Parma Heights',
'Penn Hills Township': 'Penn Hills',
'Pheonix Az': 'Phoenix',
'Pheonix': 'Phoenix',
'Phx': 'Phoenix',
'Prã@Vost': 'Prevost',
'Regional Municipality Of York': 'York',
'Richmond Height': 'Richmond Heights',
'Richmond Hts': 'Richmond Heights',
'Richmond Hil': 'Richmond Hill',
'Rocky View': 'Rocky View County',
'Rocky View No. 44': 'Rocky View County',
'Rosemã"Re': 'Rosemere',
'Ross': 'Ross Township',
'Saint-Jã@Rã'Me': 'Saint-Jerome',
'Saint-Lã@Onard': 'Saint-Leonard',

```

```

'Saint-Pierre-De-Vã@Ronne-Ã -Pike-River': 'Saint-Pierre-De-Veronne-A-Pike-River',
'Saint-Sauveur-Des-Monts': 'Saint-Sauveur',
'Sainte-Adã"Le': 'Sainte-Adele',
'Sainte-Dorothã@E': 'Sainte-Dorothee',
'Sainte-Geneviã"Ve': 'Sainte-Genevieve',
'Sainte-Thã@Rã"Se': 'Sainte-Therese',
'Scarobrough': 'Scarborough',
'Schottsdale': 'Scottsdale',
'Sheffield Lake': 'Sheffield',
'Sheffield Village': 'Sheffield',
'St Leonard': 'Saint-Leonard',
'St-Leonard': 'Saint-Leonard',
'St-Jerome': 'Saint-Jerome',
'St-Laurent': 'Saint-Laurent',
'Ste-Therese-De-Blainville': 'Sainte-Therese',
'Summerlin': 'Summerlin South',
'Sun Praie': 'Sun Prairie',
'Thornhil': 'Thornhill',
'Township Of Concord': 'Concord',
'Upper St Clair': 'Upper Saint Clair',
'Vaudreuil': 'Vaudreuil-Dorion',
'Venise-En-Quã@Bec': 'Venise-en-Quebec',
'Ville Mont-Royal': 'Mont-Royal',
'Wilkins Township': 'Wilkins Township',
'Willoughby Hills': 'Willoughby',
'Ãžle-Des-Soeurs': 'Ile-des-Soeurs'
}

df_restaurants['ville'] = df_restaurants['ville'].replace(corrections_villes)

```

Dataset nettoyé :

In [409...

df_restaurants

Out[409...

	restaurant_id	nom	moyenne_etoiles	ville	ferme
0	ICwqJWMxvIUQt1Re_tDn4w	Denny's	2.5	Las Vegas	False
1	pd0v6sOqpLhFJ7mkplaixw	Ike's Love & Sandwiches	4.0	Phoenix	False
2	0vhi__HtC2L4-vScgDFdFw	Midori Japanese Cafe	3.5	Calgary	False
3	t65yfB9v9fqlhAkLnnUXdg	Pho U	3.5	Toronto	False
4	i7_JPit-2kAbtRTLkic2jA	John & Sons Oyster House	4.0	Toronto	False
...	...	...	...	...	...
34263	cjZfgcQwA6KmQ_ANWKN2aw	Bruegger's Bagels	3.5	McMurray	True
34264	Hq2edcOTjse7wjK2CwBijQ	Bistro Pointe-Claire	3.5	Pointe-Claire	True
34265	7KlpgRjjAmVabPzxcExs0g	Taco Mex	4.0	Phoenix	True
34266	0fY-zYyP2fDmp2YXFsuNTg	Gotham Provisions Company	4.0	Sun Prairie	True
34267	cfrN6-IQC-dzjBtNBjefpQ	Rusty Nail Banquet Hall	2.5	Kent	True

34268 rows × 5 columns

Servies

Vérifications :

In [410...

df_services

Out[410...

	restaurant_id	emporter	livraison	bon_pour_groupes	bon_pour_enfai
0	ICwqJWMxvIUQt1Re_tDn4w	1	0	1	
1	pd0v6sOqpLhFJ7mkplaixw	1	1	1	
2	0vhi__HtC2L4-vScgDFdFw	1	0	0	
3	t65yfB9v9fqIhAkLnnUXdg	1	1	0	
4	i7_JPit-2kAbtRTLkic2jA	1	0	1	
...	...	...	...	...	...
34263	cjZfgcQwA6KmQ_ANWKN2aw	1	0	1	
34264	Hq2edcOTjse7wjK2CwBijQ	0	0	0	
34265	7KlpgRjjAmVabPzxcExs0g	1	0	1	
34266	0fY-zYyP2fDmp2YXFsuNTg	1	0	1	
34267	cfrN6-IQC-dzjBtNBjefpQ	0	0	0	

34268 rows × 8 columns



In [411...

df_services.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34268 entries, 0 to 34267
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   restaurant_id         34268 non-null  object
1   emporter              34268 non-null  int64
2   livraison             34268 non-null  int64
3   bon_pour_groupes      34268 non-null  int64
4   bon_pour_enfants      34268 non-null  int64
5   reservation           34268 non-null  int64
6   prix                  34268 non-null  int64
7   terrasse              34268 non-null  int64
dtypes: int64(7), object(1)
memory usage: 2.1+ MB
```

In [412...

df_services.describe(include='all')

Out[412...

	restaurant_id	emporteur	livraison	bon_pour_groupes	bon_pou
count	34268	34268.000000	34268.000000	34268.000000	34268.000000
unique	34268	NaN	NaN	NaN	NaN
top	ICwqJWMxvIUQt1Re_tDn4w	NaN	NaN	NaN	NaN
freq	1	NaN	NaN	NaN	NaN
mean	NaN	0.810173	0.208737	0.742500	0.742500
std	NaN	0.392170	0.406412	0.437263	0.437263
min	NaN	0.000000	0.000000	0.000000	0.000000
25%	NaN	1.000000	0.000000	0.000000	0.000000
50%	NaN	1.000000	0.000000	1.000000	1.000000
75%	NaN	1.000000	0.000000	1.000000	1.000000
max	NaN	1.000000	1.000000	1.000000	1.000000

In [413...

```
df_services.isna().sum()
```

Out[413...

```
restaurant_id    0
emporteur        0
livraison        0
bon_pour_groupes 0
bon_pour_enfants 0
reservation      0
prix            0
terrasse         0
dtype: int64
```

In [414...

```
# Vérification s'il existe des données autres que 1 ou 0
df_services[(df_services['emporteur'] != 1) & (df_services['emporteur'] != 0)
             | (df_services['livraison'] != 1) & (df_services['livraison'] != 0)
             | (df_services['bon_pour_groupes'] != 1) & (df_services['bon_pour_groupes'] != 0)
             | (df_services['bon_pour_enfants'] != 1) & (df_services['bon_pour_enfants'] != 0)
             | (df_services['reservation'] != 1) & (df_services['reservation'] != 0)
             | (df_services['terrasse'] != 1) & (df_services['terrasse'] != 0)]
```

Out[414...

```
restaurant_id  emporteur  livraison  bon_pour_groupes  bon_pour_enfants  reservation  pri
```

Nettoyage :

In [415...

```
# Conversion vers le types approprié
df_services['restaurant_id'] = df_services['restaurant_id'].astype('string')
df_services['emporteur'] = df_services['emporteur'].astype(bool)
df_services['livraison'] = df_services['livraison'].astype(bool)
df_services['bon_pour_groupes'] = df_services['bon_pour_groupes'].astype(bool)
```

```
df_services['bon_pour_enfants'] = df_services['bon_pour_enfants'].astype(bool)
df_services['reservation'] = df_services['reservation'].astype(bool)
df_services['terrasse'] = df_services['terrasse'].astype(bool)
df_services.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 34268 entries, 0 to 34267
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   restaurant_id         34268 non-null  string
1   emporteur              34268 non-null  bool
2   livraison              34268 non-null  bool
3   bon_pour_groupes       34268 non-null  bool
4   bon_pour_enfants       34268 non-null  bool
5   reservation            34268 non-null  bool
6   prix                   34268 non-null  int64
7   terrasse               34268 non-null  bool
dtypes: bool(6), int64(1), string(1)
memory usage: 736.4 KB
```

Utilisateurs

Vérifications :

In [416... df_utilisateurs

Out[416...

	utilisateur_id	nom	elite	nb_avis	nb_avis_mention
0	I6BmjZMeQD3rDxWUbiAiow	Rashmi	1	95	84
1	4XChL029mKr5hydo79Ljxg	Jenna	0	33	48
2	bc8C_eETBWL0olvFSJJd0w	David	0	16	28
3	MM4RJAeH6yuaN8oZDSt0RA	Nancy	1	361	1114
4	TEtzbpgA2BFBrC0y0sCbfbw	Keane	1	1122	13311
...	...	...	...	...	...
748242	qIKRzgQwcLsRbcjFYD1jkg	Joel	0	3	0
748243	vyK8txmse-HV7hAKCvuMQ	Keith	0	10	4
748244	gCfFBY-cO1Y0TxxKX89-ig	Maryorie	0	1	0
748245	Z2uEg8DIk1twiNC6NV7e9w	Adriana	0	1	1
748246	ib_elCBhGOamS0AALkh4Og	Lee	0	4	8

748247 rows × 5 columns

In [417... df_utilisateurs.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 748247 entries, 0 to 748246
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   utilisateur_id        748247 non-null  object
1   nom                    748239 non-null  object
2   elite                  748247 non-null  int64
3   nb_avis                748247 non-null  int64
4   nb_avis_mention       748247 non-null  int64
dtypes: int64(3), object(2)
memory usage: 28.5+ MB
```

```
In [418... df_utilisateurs.describe(include='all')
# Tous des ID uniques
# Minimum de 0 et maximum de 1 pour elite
# Ne semble pas avoir des nombres négatifs
```

```
Out[418...
```

	utilisateur_id	nom	elite	nb_avis	nb_avis_mention
count	748247	748239	748247.000000	748247.000000	748247.000000
unique	748247	72126	NaN	NaN	NaN
top	I6BmjZMeQD3rDxWUbiAiw	John	NaN	NaN	NaN
freq	1	6668	NaN	NaN	NaN
mean	NaN	NaN	0.072799	34.38974	67.922637
std	NaN	NaN	0.259807	105.32548	667.026047
min	NaN	NaN	0.000000	0.000000	0.000000
25%	NaN	NaN	0.000000	3.000000	1.000000
50%	NaN	NaN	0.000000	9.000000	5.000000
75%	NaN	NaN	0.000000	24.000000	22.000000
max	NaN	NaN	1.000000	13278.000000	154202.000000

In [419... df_utilisateurs.isna().sum()

```
Out[419... utilisateur_id    0
nom                8
elite              0
nb_avis            0
nb_avis_mention    0
dtype: int64
```

In [420... df_utilisateurs[df_utilisateurs['utilisateur_id'] == '']

```
Out[420... utilisateur_id  nom  elite  nb_avis  nb_avis_mention
```

```
In [421... # Vérification s'il existe des données autres que 1 ou 0
df_utilisateurs[(df_utilisateurs['elite'] != 1) & (df_utilisateurs['elite'] != 0)]

Out[421...      utilisateur_id  nom  elite  nb_avis  nb_avis_mention

In [422... # Vérification s'il existe des nombres négatifs
df_utilisateurs[df_utilisateurs['nb_avis'] < 0]

Out[422...      utilisateur_id  nom  elite  nb_avis  nb_avis_mention

In [423... # Vérification s'il existe des nombres négatifs
df_utilisateurs[df_utilisateurs['nb_avis_mention'] < 0]

Out[423...      utilisateur_id  nom  elite  nb_avis  nb_avis_mention

In [424... # Vérification s'il existe un ID dupliqué
df_utilisateurs['utilisateur_id'].duplicated().sum()

Out[424... np.int64(0)
```

Nettoyages :

```
In [425... # Même si Les noms sont manquants, on a Les IDs et Le nombre d'avis
df_utilisateurs['nom'] = df_utilisateurs['nom'].fillna('Inconnu')
df_utilisateurs[df_utilisateurs['nom'] == 'Inconnu']

Out[425...      utilisateur_id  nom  elite  nb_avis  nb_avis_mention

      87688  GiReVCFFK4WuhocoxDmOvw  Inconnu    0         5         0
      161155    aahxmN1tsvlufuU_y4OT1A  Inconnu    0        23        61
      207141    UgeQ4jZUvh8aknnOmvKRwg  Inconnu    0         9        11
      237770    7UIRHLbYWR7RrOp1pHDJw  Inconnu    1       486       945
      245555    degkhE1YTY9x3Vt2OMEZbw  Inconnu    0        15        10
      288643    OmBde8ljHEcO0QwS_jl5sA  Inconnu    0         9         3
      369701    UxQyVv6m5N0Eb2hBp7VDqw  Inconnu    0         4         0
      519971    nLJJAZM8CPA3W56cpZ9gxw  Inconnu    0         3         6

In [426... # Conversion vers le types approprié
df_utilisateurs['utilisateur_id'] = df_utilisateurs['utilisateur_id'].astype('strin
df_utilisateurs['nom'] = df_utilisateurs['nom'].astype('string')
df_utilisateurs['elite'] = df_utilisateurs['elite'].astype(bool)
df_utilisateurs.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 748247 entries, 0 to 748246
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   utilisateur_id        748247 non-null  string
1   nom                   748247 non-null  string
2   elite                 748247 non-null  bool
3   nb_avis               748247 non-null  int64
4   nb_avis_mention       748247 non-null  int64
dtypes: bool(1), int64(2), string(2)
memory usage: 23.5 MB
```

Dataset nettoyé :

In [427...

df_utilisateurs

Out[427...

	utilisateur_id	nom	elite	nb_avis	nb_avis_mention
0	I6BmjZMeQD3rDxWUbiAiow	Rashmi	True	95	84
1	4XChL029mKr5hydo79Ljxg	Jenna	False	33	48
2	bc8C_eETBWL0olvFSJd0w	David	False	16	28
3	MM4RJAeH6yuaN8oZDSt0RA	Nancy	True	361	1114
4	TEtzbpgA2BFBrC0y0sCbfw	Keane	True	1122	13311
...	...	...	...	...	...
748242	qIKRzgQwcLsRbcjFYD1jkg	Joel	False	3	0
748243	vyK8txmse-HV7hAKCvuMQ	Keith	False	10	4
748244	gCfFBY-cO1Y0TxxKX89-ig	Maryorie	False	1	0
748245	Z2uEg8DIk1twiNC6NV7e9w	Adriana	False	1	1
748246	ib_eICBhGOamS0AALkh4Og	Lee	False	4	8

748247 rows × 5 columns

Création du Dataset Business

In [428...

```
df_business = df_restaurants[['restaurant_id', 'nom', 'ville', 'moyenne_etoiles']].
review_count = df_avis.groupby('restaurant_id')['avis_id'].count().reset_index(name=

df_business = pd.merge(df_business, review_count, how='left', on='restaurant_id')
df_business = pd.merge(df_business, df_categories.groupby('restaurant_id')['categor
df_business = df_business.rename(columns={'restaurant_id': 'business_id', 'categori
```

```
df_business['categories'] = df_business['categories'].astype('string')
df_business
```

Out[428...

	business_id	nom	ville	etoiles	review_count	categories
0	lCwqJWMxvIUQt1Re_tDn4w	Denny's	Las Vegas	2.5	72	Breakfast Brunch Diner American (Traditional)
1	pd0v6sOqpLhFJ7mkplaixw	Ike's Love & Sandwiches	Phoenix	4.0	108	Sandwiches Gluten Free Vegetarian Vegan
2	0vhi__HtC2L4-vScgDFdFw	Midori Japanese Cafe	Calgary	3.5	49	Japanese
3	t65yfB9v9fqIhAkLnnUXdg	Pho U	Toronto	3.5	36	Vietnamese Chinese
4	i7_JPit-2kAbtRTLkic2jA	John & Sons Oyster House	Toronto	4.0	88	Seafood
...	...	...	...	...	...	...
34263	cjZfgcQwA6KmQ_ANWKN2aw	Bruegger's Bagels	McMurray	3.5	6	Sandwiches Breakfast Brunch Bagels
34264	Hq2edcOTjse7wjK2CwBijQ	Bistro Pointe- Claire	Pointe- Claire	3.5	11	Breakfast Brunch Canadian (New)
34265	7KlpgRjjAmVabPzxcExs0g	Taco Mex	Phoenix	4.0	11	Mexican
34266	0fY-zYyP2fDmp2YXFsuNTg	Gotham Provisions Company	Sun Prairie	4.0	18	Sandwiches Soups Bagels
34267	cfrN6-IQC-dzjBtNBjefpQ	Rusty Nail Banquet Hall	Kent	2.5	3	American (New)

34268 rows × 6 columns



Jointure entre les différents Data sets

```
In [429... data = DataFrame(df_restaurants)
data = data.drop(["ferme"], axis = 1)
```

Préparer review count

```
In [430... review_count_join = (df_avis
    .groupby("restaurant_id")
    .size()
    .reset_index(name = "review_count_total"))
```

```
In [431... positive_reviews = (df_avis[df_avis["etoiles"] >= 4]
    .groupby("restaurant_id")
    .size()
    .reset_index(name = "review_positive"))
```

```
In [432... review = merge(review_count_join,
    positive_reviews,
    left_on= "restaurant_id",
    right_on="restaurant_id",
    how="inner")
```

```
In [433... data = data.merge(review,
    left_on = "restaurant_id",
    right_on = "restaurant_id",
    how = "inner")
```

```
In [434... data["positive_ratio"] = floor((data["review_positive"] / data["review_count_total"]
```

Préparations checkin total

```
In [435... checkins_total = (df_checkin.groupby("restaurant_id")
    .size()
    .reset_index(name="checkins_total"))
```

```
In [436... data = data.merge(checkins_total,
    left_on = "restaurant_id",
    right_on = "restaurant_id",
    how = "inner")
```

```
In [437... name_counts = data["nom"].value_counts()
```

```
In [438... data["is_chain"] = data["nom"].map(lambda x: name_counts[x] >= 3)
```

```
In [439... prix_moyen = (df_services
    .groupby("restaurant_id")["prix"]
    .mean()
    .reset_index(name="prix_moyen"))
```



```
In [440...] data = data.merge(prix_moyen,
                        left_on = "restaurant_id",
                        right_on = "restaurant_id",
                        how = "left")
```

```
In [441...] df_elite = df_utilisateurs[df_utilisateurs["elite"].notna()]
```

```
In [442...] df_elite = df_utilisateurs[df_utilisateurs["elite"] != "[]"]
```

```
In [443...] elite_id = df_elite[["utilisateur_id"]]
```

```
In [444...] elite_id
```

```
Out[444...]      utilisateur_id
```

0	I6BmjZMeQD3rDxWUbiAiow
1	4XChL029mKr5hydo79Ljxg
2	bc8C_eETBWL0olvFSJJd0w
3	MM4RJAeH6yuaN8oZDSt0RA
4	TEtzbpgA2BFBrC0y0sCbfbw
...	...
748242	qIKRzgQwcLsRbcjFYD1jkg
748243	vyK8txmse-HV7hAKCvuMQ
748244	gCfFBY-cO1Y0TxxKX89-ig
748245	Z2uEg8DIk1twiNC6NV7e9w
748246	ib_eICBhGOamS0AALkh4Og

748247 rows × 1 columns

```
In [445...] elite_reviews = df_avis.merge(elite_id,
                                           on="utilisateur_id",
                                           how="inner")
```

```
In [446...] elite_users_count = (elite_reviews
                                  .groupby("restaurant_id")["utilisateur_id"]
                                  .nunique()
                                  .reset_index(name="elite_users_count"))
```

```
In [447...] data = data.merge(elite_users_count,
                              on="restaurant_id",
                              how="left")
```

```
In [448...] for day in days:
              df_horaires[day] = df_horaires[day].apply(parse_hours)
```

```
In [449... df_horaires["avg_open_hours"] = df_horaires[days].mean(axis=1)
```

```
In [450... df_horaires["avg_open_hours"] = floor(df_horaires["avg_open_hours"] * 100) / 100
```

```
In [451... data = data.merge(df_horaires[["restaurant_id", "avg_open_hours"]],
    on="restaurant_id",
    how="left")
```

```
In [452... data.to_csv(export_path)
```

Graphiques

Les catégories les plus appréciées par villes

Préparation des données

```
In [453... df_categories_aimees = df_categories.merge(df_restaurants[["restaurant_id", "ville"]],
df_categories_aimees = df_categories_aimees.merge(review_count[["restaurant_id", "r

df_categories_aimees = (df_categories_aimees.groupby(["ville", "categorie"]).agg(
    etoiles=('moyenne_etoiles', 'mean'),
    avis=('review_count', 'sum'))).reset_index()

liste_villes = ['Montreal', 'Laval', 'Toronto', 'Longueuil', 'Brampton', 'Etobicoke

# df_categories_aimees[df_categories_aimees['ville'] == 'Toronto'].head(30)
```

Normalisation des données

```
In [454... avis_ville_moyenne = df_categories_aimees.groupby('ville')['avis'].transform('mean')
df_categories_aimees = df_categories_aimees[df_categories_aimees['avis'] >= avis_vi

df_top3_par_villes = (df_categories_aimees.sort_values(['ville', 'etoiles'], ascend
```

```
In [455... df_top3_par_villes[df_top3_par_villes['ville'] == 'Montreal']
```

```
Out[455...
      ville  categorie  etoiles  avis
7496  Montreal  Cheese Shops  4.318182  2504
7591  Montreal    Meat Shops  4.300000   687
7457  Montreal    Bakeries  4.158333  1219
```

Préparation du graphique

```
In [456... villes = df_top3_par_villes[df_top3_par_villes['ville'].isin(liste_villes)]

fig, axes = plt.subplots(
    nrows=len(liste_villes),
    ncols=1,
    figsize=(10, 3 * len(liste_villes)),
    constrained_layout=True
)

for ax, ville in zip(axes, liste_villes):

    subset = villes[villes['ville'] == ville]
    subset = subset.sort_values('etoiles')

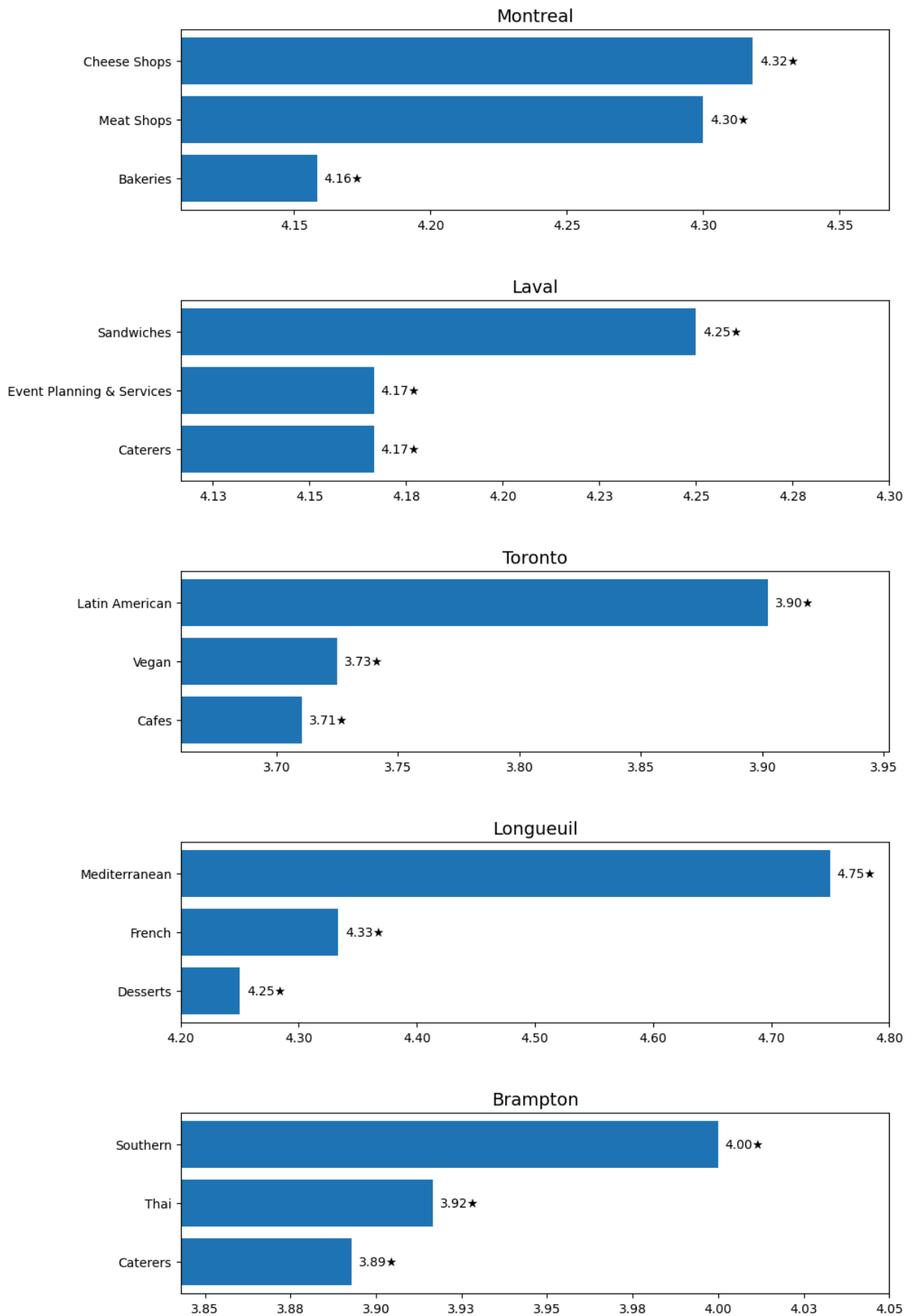
    ax.barh(subset['categorie'], subset['etoiles'])
    ax.set_title(f'\n\n{ville}', fontsize=14)
    ax.xaxis.set_major_formatter(plt.FormatStrFormatter('%.2f'))

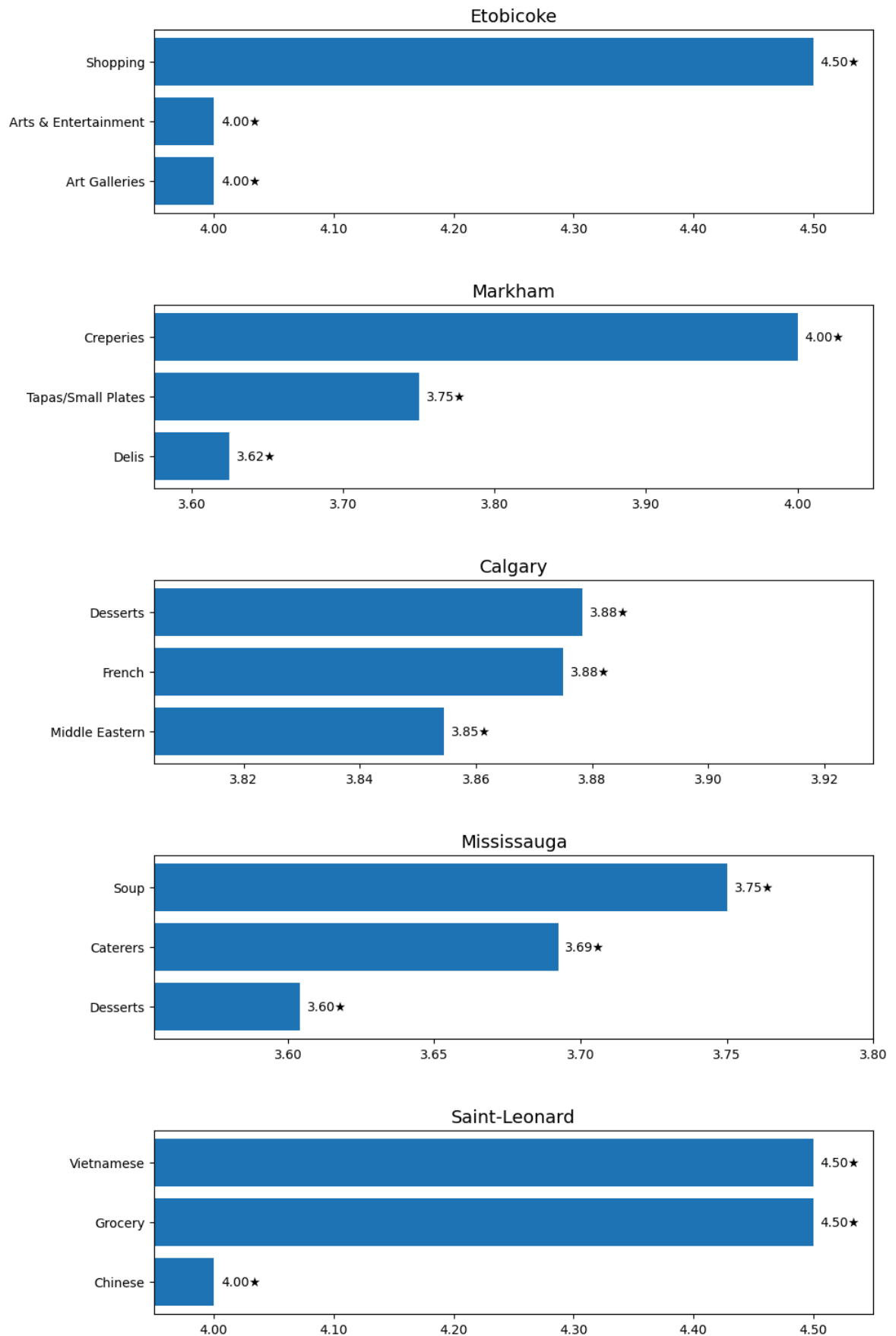
    min_etoiles = subset['etoiles'].min() - 0.05
    max_etoiles = subset['etoiles'].max() + 0.05
    ax.set_xlim(min_etoiles, max_etoiles)

    for i, (etoiles, avis) in enumerate(zip(subset['etoiles'], subset['avis'])):
        ax.text(etoiles + (max_etoiles - min_etoiles) * 0.01, i, f'{etoiles:.2f}★',

fig.suptitle('Top 3 des types de commerces les plus aimées (par ville)', fontsize=2
plt.show()
```

Top 3 des types de commerces les plus aimées (par ville)





Préparation Graphique interactif

In [457...

```

numeric_columns = data.select_dtypes('number').columns

x_dropdown = Dropdown(options=numeric_columns, description='X')
y_dropdown = Dropdown(options=numeric_columns, description='Y')
z_dropdown = Dropdown(options=numeric_columns, description='Z')

city_dropdown = Dropdown(
    options=['(All)'] + sorted(data['ville'].unique()),
    description='Ville'
)

@interact(x=x_dropdown, y=y_dropdown, z=z_dropdown, city=city_dropdown)
def update_plot(x, y, z, city):
    ville = data if city == '(All)' else data[data['ville'] == city]

    fig = figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection='3d')

    ax.scatter(ville[x], ville[y], ville[z], s=40, alpha=0.8)

    ax.set_xlabel(x)
    ax.set_ylabel(y)
    ax.set_zlabel(z)
    ax.set_title(f"Scatter 3D : {x} vs {y} vs {z}\nVille : {city}")

    show()

```

```

interactive(children=(Dropdown(description='X', options=('moyenne_etoiles', 'review_
count_total', 'review_posi...

```